Homework - 6

Repository: https://github.com/Ani2508/data260-3209

Tagged commit (hw6): to be filled after tagging hw6

Domain: Clinical Trial Listing

Section 0 - Personal Configuration

Value	Result
SID4	3209
PORT_BASE	8509 (8000 + 3209 mod 900)
PREFIX	s3209
SEED	3209
VERIFY_SEED	263209
DOMAIN_ID	1 - Clinical trial listings

Environment

Hardware	Apple MacBook Pro, Apple M5 Pro chip
RAM	48 GB
OS	macOS
Local model	llama3.1:8b (Ollama), temperature 0, seed 3209
Embedding model	nomic-embed-text (Ollama, 768 dimensions)
Database	MongoDB (s3209_doc); HW5 MySQL routes unchanged

Part 1 - MongoDB Document API

Database s3209_doc, collection trials: a document view of a clinical trial listing. The API runs on PORT_BASE 8509 next to the HW5 MySQL routes.

I. Database

Field	Type	Rule
title	string	required, 1-100 characters
description	string / null	optional, max 2000 characters
status	enum	recruiting, active, completed, suspended, withdrawn (default recruiting)
phase	enum	phase_1, phase_2, phase_3, phase_4, observational
start_date	date	required, ISO 8601
created_at / updated_at	date	set automatically by the API

Code 1 - Server configuration (config.py)

Personal values and MongoDB settings; database name built from PREFIX.

```
 9
10    # Personal configuration (Section 0)
11    SID4 = 3209
12    PORT_BASE = 8000 + (SID4 % 900)          # 8509
13    PREFIX = f"s{SID4}"                      # s3209
14    SEED = SID4
15    VERIFY_SEED = 260000 + SID4              # 263209
16
17    # MongoDB
18    MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
19    MONGO_DB = os.getenv("MONGO_DB", f"{PREFIX}_doc")   # s3209_doc
```

Code 2 - Database validation rules (mongo_db.py)

$jsonSchema validator: required fields, length limits and enum constraints enforced by MongoDB.

```
10    TRIAL_STATUSES = ["recruiting", "active", "completed", "suspended", "withdrawn"]
11    TRIAL_PHASES = ["phase_1", "phase_2", "phase_3", "phase_4", "observational"]
12
13    TRIALS_VALIDATOR = {"$jsonSchema": {
14        "bsonType": "object",
15        "required": ["title", "status", "phase", "start_date", "created_at", "updated_at"],
16        "properties": {
17            "title": {"bsonType": "string", "minLength": 1, "maxLength": 100},
18            "description": {"bsonType": ["string", "null"], "maxLength": 2000},
19            "status": {"enum": TRIAL_STATUSES},
20            "phase": {"enum": TRIAL_PHASES},
21            "start_date": {"bsonType": "date"},
22            "created_at": {"bsonType": "date"},
23            "updated_at": {"bsonType": "date"},
24        },
25    }}
```

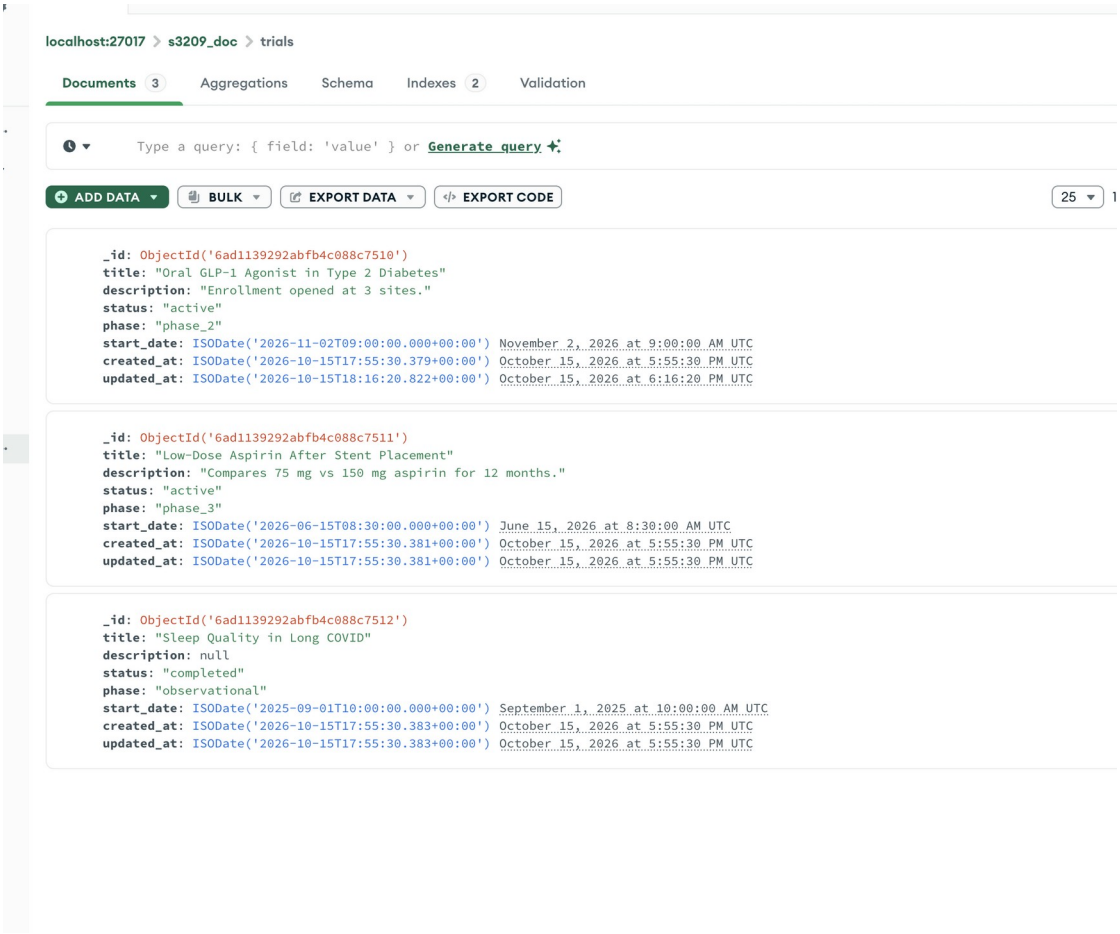
Code 3 - Collection setup (mongo_db.py)

init_db() creates the collections with their validators and indexes at startup.

```
83    def init_db():
84        """Create collections, validators and indexes (idempotent)."""
85        db = get_db()
86        _ensure(db, "trials", TRIALS_VALIDATOR)
87        _ensure(db, "messages", MESSAGES_VALIDATOR)
88        _ensure(db, "summaries", SUMMARIES_VALIDATOR)
89        _ensure(db, "episodes", EPISODES_VALIDATOR)
90        db.trials.create_index([("status", ASCENDING), ("start_date", ASCENDING)])
91        db.messages.create_index([("user_id", ASCENDING), ("session_id", ASCENDING), ("created_at", DESCENDING)])
92        db.summaries.create_index([("user_id", ASCENDING), ("scope", ASCENDING), ("created_at", DESCENDING)])
93        db.episodes.create_index([("user_id", ASCENDING), ("created_at", DESCENDING)])
94        return db
95
```

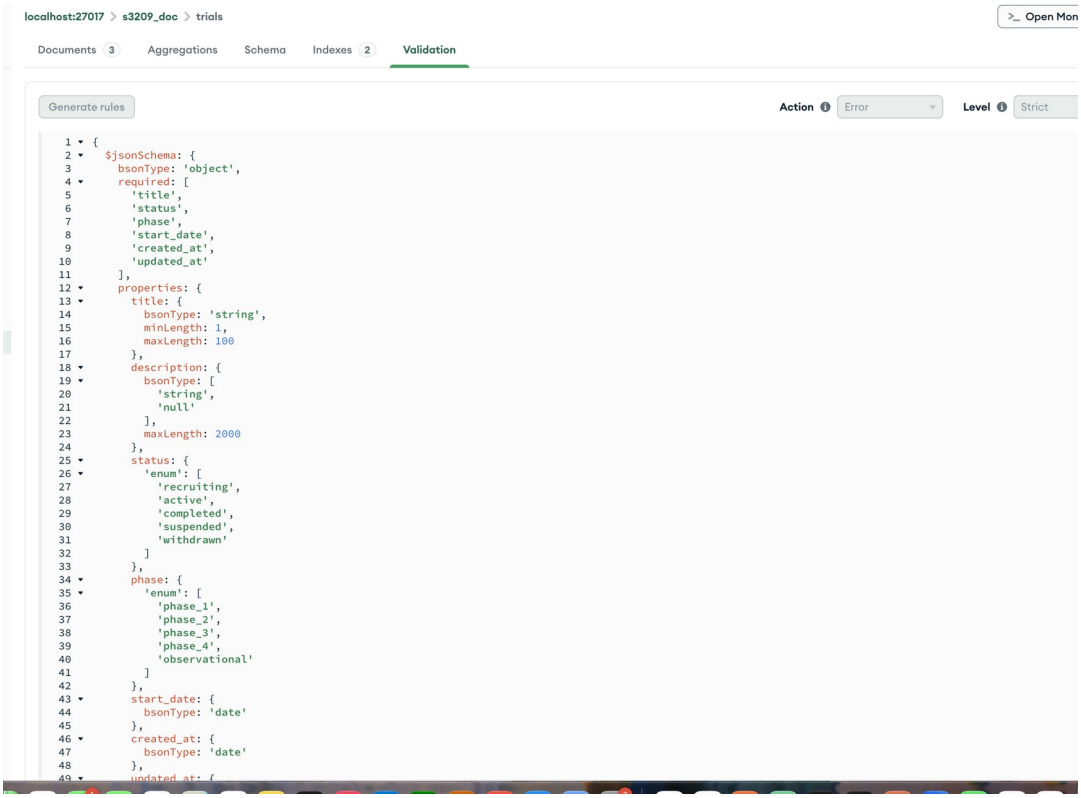
Output 1 - trials collection in MongoDB Compass

Three sample trials stored in s3209_doc.trials; description is null where omitted.



Output 2 - Validation rules in MongoDB Compass

The $jsonSchema rules applied on the collection (action Error, level Strict).



II. API Implementation

File	Role
config.py	Server configuration (port, database, CORS origins)
mongo_db.py	PyMongo connection, validators, indexes
doc_schemas.py	Pydantic models and enums
doc_trials.py	Route handlers for CRUD
main.py	App setup, CORS middleware, error handlers

Code 4 - CRUD route handlers (doc_trials.py)

POST, GET all, GET by id, PUT and DELETE; 404 when the id does not exist.

```python

@router.post("", response_model=DocTrialOut, status_code=status.HTTP_201_CREATED)
def create_trial(body: DocTrialCreate):
    now = _now()
    doc = body.model_dump(mode="python")
    doc["status"], doc["phase"] = body.status.value, body.phase.value
    doc.update(created_at=now, updated_at=now)
    result = get_db().trials.insert_one(doc)
    doc["_id"] = result.inserted_id
    return _out(doc)


@router.get("", response_model=list[DocTrialOut])
def list_trials(status_: str | None = Query(None, alias="status"),
                limit: int = Query(50, ge=1, le=200)):
    query = {"status": status_} if status_ else {}
    docs = get_db().trials.find(query).sort("start_date", 1).limit(limit)
    return [_out(d) for d in docs]

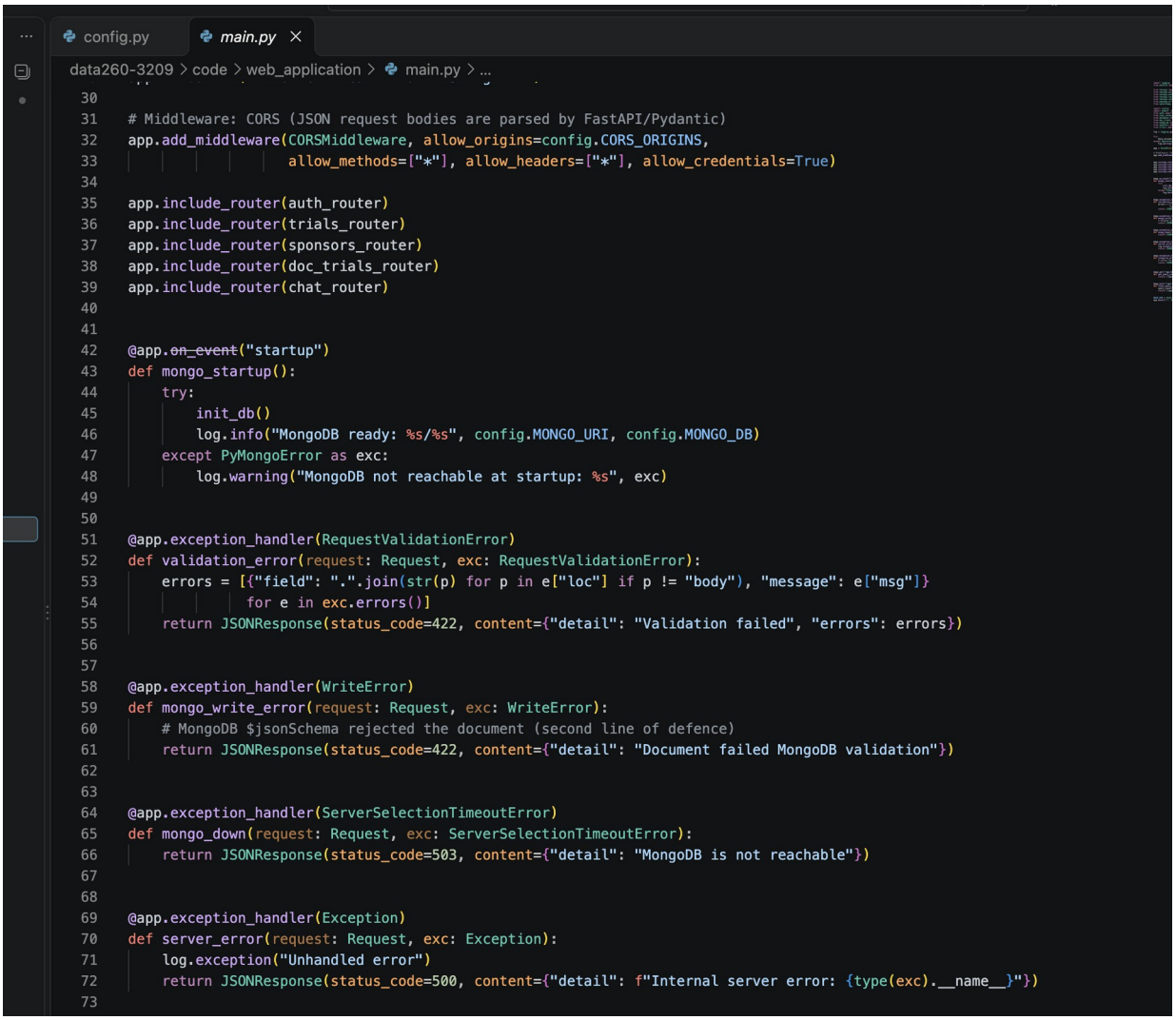

@router.get("/{trial_id}", response_model=DocTrialOut)
def get_trial(trial_id: str):
    doc = get_db().trials.find_one({"_id": _oid(trial_id)})
    if not doc:
        raise HTTPException(status_code=404, detail=f"Trial {trial_id} not found")
    return _out(doc)


@router.put("/{trial_id}", response_model=DocTrialOut)
def update_trial(trial_id: str, body: DocTrialUpdate):
    changes = {k: (v.value if hasattr(v, "value") else v)
               for k, v in body.model_dump(exclude_unset=True).items()}
    if not changes:
        raise HTTPException(status_code=400, detail="No fields to update")
    if changes.get("title", "x") is None:
        raise HTTPException(status_code=422, detail="title cannot be null")
    changes["updated_at"] = _now()
    doc = get_db().trials.find_one_and_update({"_id": _oid(trial_id)}, {"$set": changes},
                                              return_document=ReturnDocument.AFTER)
    if not doc:
        raise HTTPException(status_code=404, detail=f"Trial {trial_id} not found")
    return _out(doc)


@router.delete("/{trial_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_trial(trial_id: str):
    result = get_db().trials.delete_one({"_id": _oid(trial_id)})
    if result.deleted_count == 0:
        raise HTTPException(status_code=404, detail=f"Trial {trial_id} not found")
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

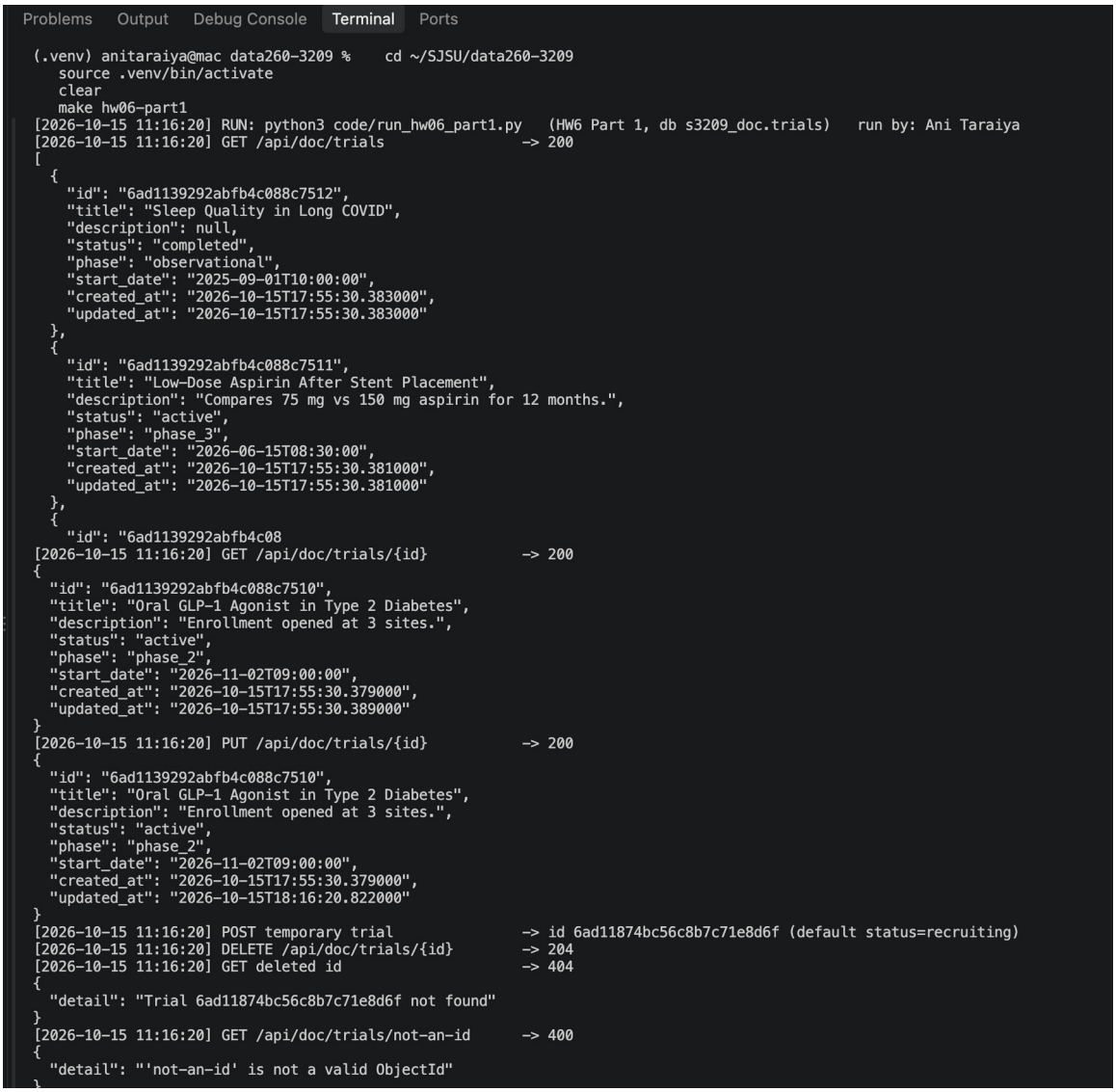
Code 5 - CORS middleware and error handlers (main.py)

CORS middleware; clear JSON errors for validation (422), MongoDB down (503) and server errors (500).



Output 3 - CRUD run (make hw06-part1)

GET all, GET by id, PUT, POST with default status, DELETE (204), then 404 and 400 for a bad id. Long JSON is trimmed by the script.



Operation	Endpoint	Status
Create	POST /api/doc/trials	201
Retrieve all	GET /api/doc/trials	200
Get one	GET /api/doc/trials/{id}	200
Update	PUT /api/doc/trials/{id}	200
Delete	DELETE /api/doc/trials/{id}	204
Deleted / unknown id	GET /api/doc/trials/{id}	404
Invalid id format	GET /api/doc/trials/not-an-id	400

III. Data Validation

Code 6 - Pydantic schemas (doc_schemas.py)

Enums for status and phase, title max 100, optional description, datetime start_date, extra fields rejected.

config.py | doc_schemas.py

data260-3209 > code > web_application > doc_schemas.py > ...

```python
class TrialStatus(str, Enum):
    recruiting = "recruiting"
    active = "active"
    completed = "completed"
    suspended = "suspended"
    withdrawn = "withdrawn"


class TrialPhase(str, Enum):
    phase_1 = "phase_1"
    phase_2 = "phase_2"
    phase_3 = "phase_3"
    phase_4 = "phase_4"
    observational = "observational"


def _not_blank(value: str) -> str:
    value = value.strip()
    if not value:
        raise ValueError("must not be blank")
    return value


class DocTrialCreate(BaseModel):
    model_config = ConfigDict(extra="forbid")
    title: str = Field(min_length=1, max_length=100, examples=["Metformin XR in Type 2 Diabetes"])
    description: Optional[str] = Field(default=None, max_length=2000)
    status: TrialStatus = TrialStatus.recruiting          # enum with default
    phase: TrialPhase                                      # fixed set of options
    start_date: datetime                                   # ISO 8601 date/time

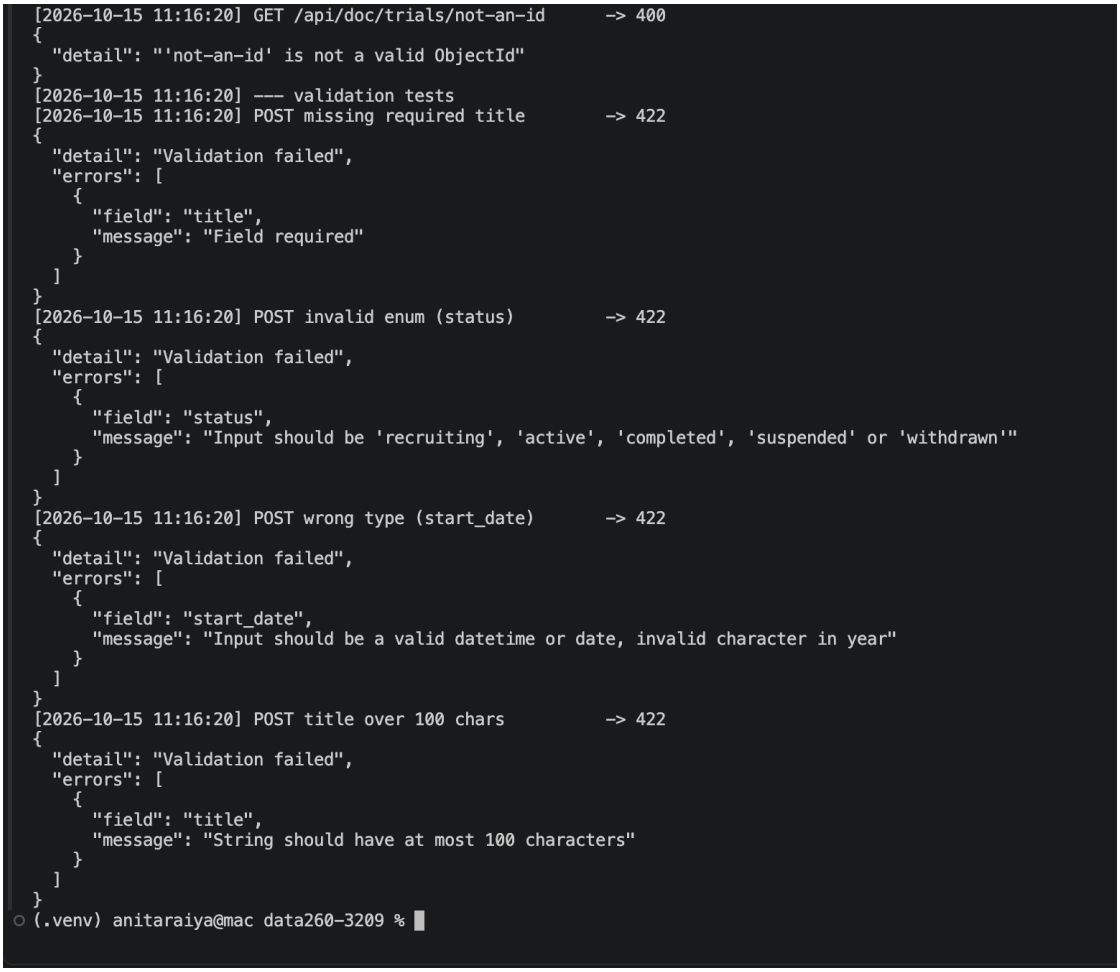
    _strip = field_validator("title")(_not_blank)


class DocTrialUpdate(BaseModel):
    """PUT body: same rules, every field optional (only sent fields change)."""
    model_config = ConfigDict(extra="forbid")
    title: Optional[str] = Field(default=None, min_length=1, max_length=100)
    description: Optional[str] = Field(default=None, max_length=2000)
    status: Optional[TrialStatus] = None
    phase: Optional[TrialPhase] = None
    start_date: Optional[datetime] = None

    @field_validator("title")
    @classmethod
    def strip_title(cls, value):
        return None if value is None else _not_blank(value)
```

Output 4 - Invalid data tests

Missing field, invalid enum, wrong date type and long title all return 422 with the field and reason.



- Two layers: Pydantic rejects bad input before it reaches the database; MongoDB $jsonSchema blocks invalid writes from any other client.
- Error format is the same for every case: detail plus a list of field and message.

Part 2 - Short-, Long-, and Episodic Memory

Same database s3209_doc with three new collections; the Part 1 trials collection is unchanged. Model llama3.1:8b through the HW1 adapter (src/model_client.py); embeddings from nomic-embed-text (768 values).

Setting	Value
SHORT_TERM_N	12 messages
SUMMARIZE_EVERY_USER_MSGS	5 (session summary)
Lifetime summary	every 2 session summaries
Episodic facts	up to 3 per message, top-k = 3 by cosine similarity

I. MongoDB Collections

Collection	Fields
messages	user_id, session_id, role (user/assistant), content, created_at
summaries	user_id, session_id (null for lifetime), scope (session/user), text, created_at
episodes	user_id, session_id, fact, importance (0..1), embedding (array of floats), created_at

Code 1 - Collection rules (mongo_db.py)

$jsonSchema validators for messages, summaries and episodes.

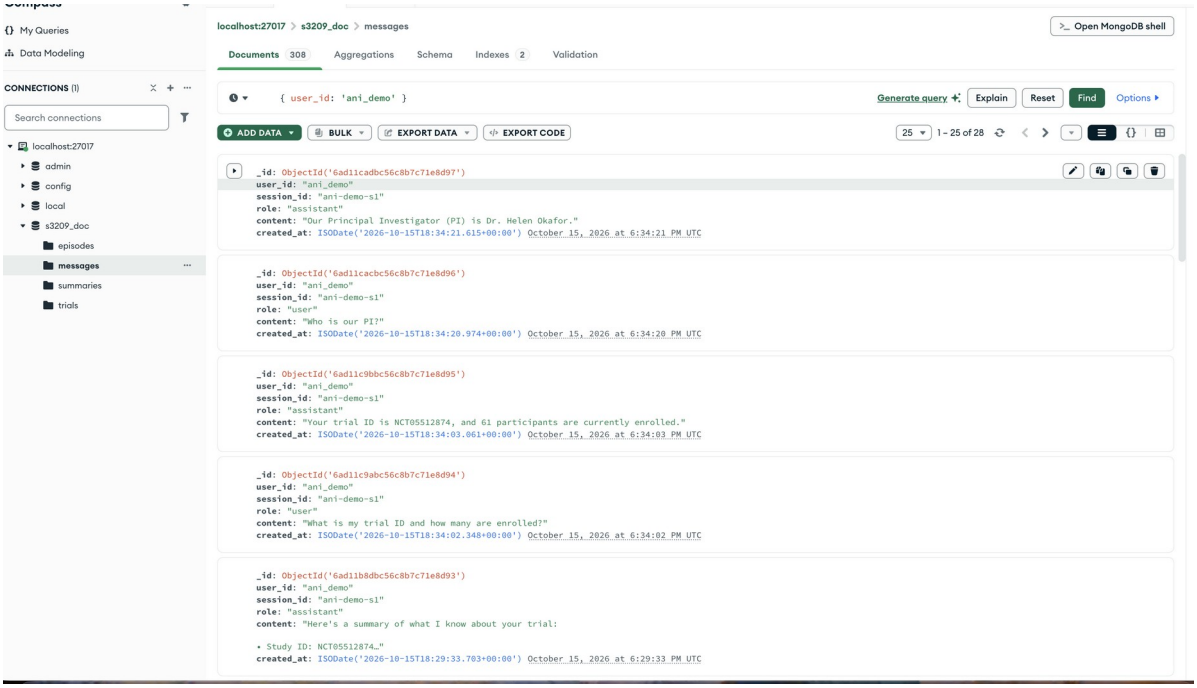
```python
MESSAGES_VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["user_id", "session_id", "role", "content", "created_at"],
    "properties": {
        "user_id": {"bsonType": "string"},
        "session_id": {"bsonType": "string"},
        "role": {"enum": ["user", "assistant"]},
        "content": {"bsonType": "string"},
        "created_at": {"bsonType": "date"},
    },
}}

SUMMARIES_VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["user_id", "scope", "text", "created_at"],
    "properties": {
        "user_id": {"bsonType": "string"},
        "session_id": {"bsonType": ["string", "null"]},
        "scope": {"enum": ["session", "user"]},
        "text": {"bsonType": "string"},
        "created_at": {"bsonType": "date"},
    },
}}

EPISODES_VALIDATOR = {"$jsonSchema": {
    "bsonType": "object",
    "required": ["user_id", "session_id", "fact", "importance", "embedding", "created_at"],
    "properties": {
        "user_id": {"bsonType": "string"},
        "session_id": {"bsonType": "string"},
        "fact": {"bsonType": "string", "maxLength": 300},
        "importance": {"bsonType": ["double", "int"], "minimum": 0, "maximum": 1},
        "embedding": {"bsonType": "array", "items": {"bsonType": "double"}},
        "created_at": {"bsonType": "date"},
    },
}}
```

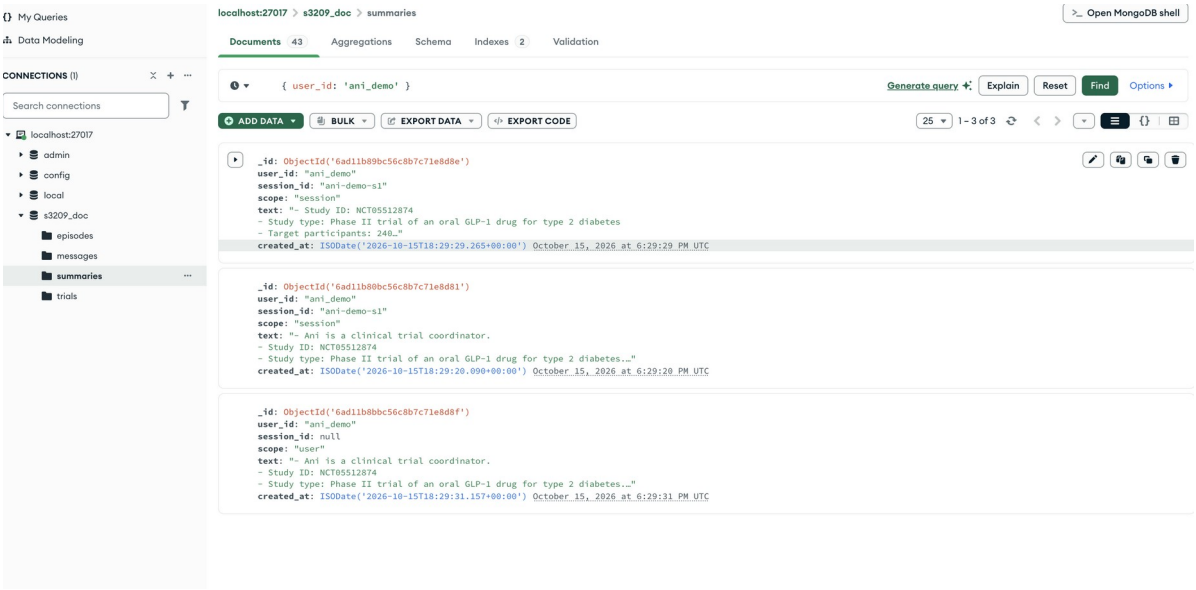
Output 1 - messages collection (Compass, user ani_demo)

Stored user and assistant turns with session_id and created_at.



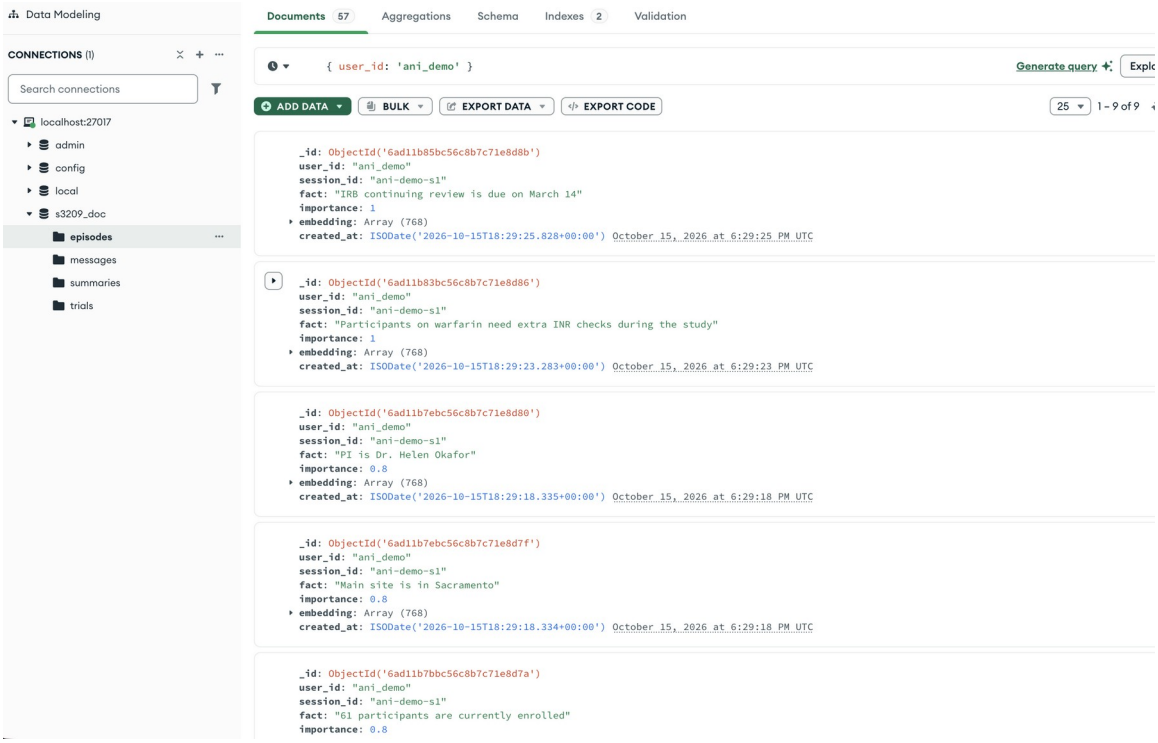
Output 2 - summaries collection (Compass, user ani_demo)

Two session summaries and one lifetime summary (scope user, session_id null).



Output 3 - episodes collection (Compass, user ani_demo)

Nine facts with importance and a 768-value embedding each.



II. FastAPI Endpoints

Code 2 - Endpoints (chat_routes.py)

POST /api/chat, GET /api/memory/{user_id} (16 messages, summaries, 20 facts), GET /api/aggregate/{user_id} (daily counts, recent summaries).

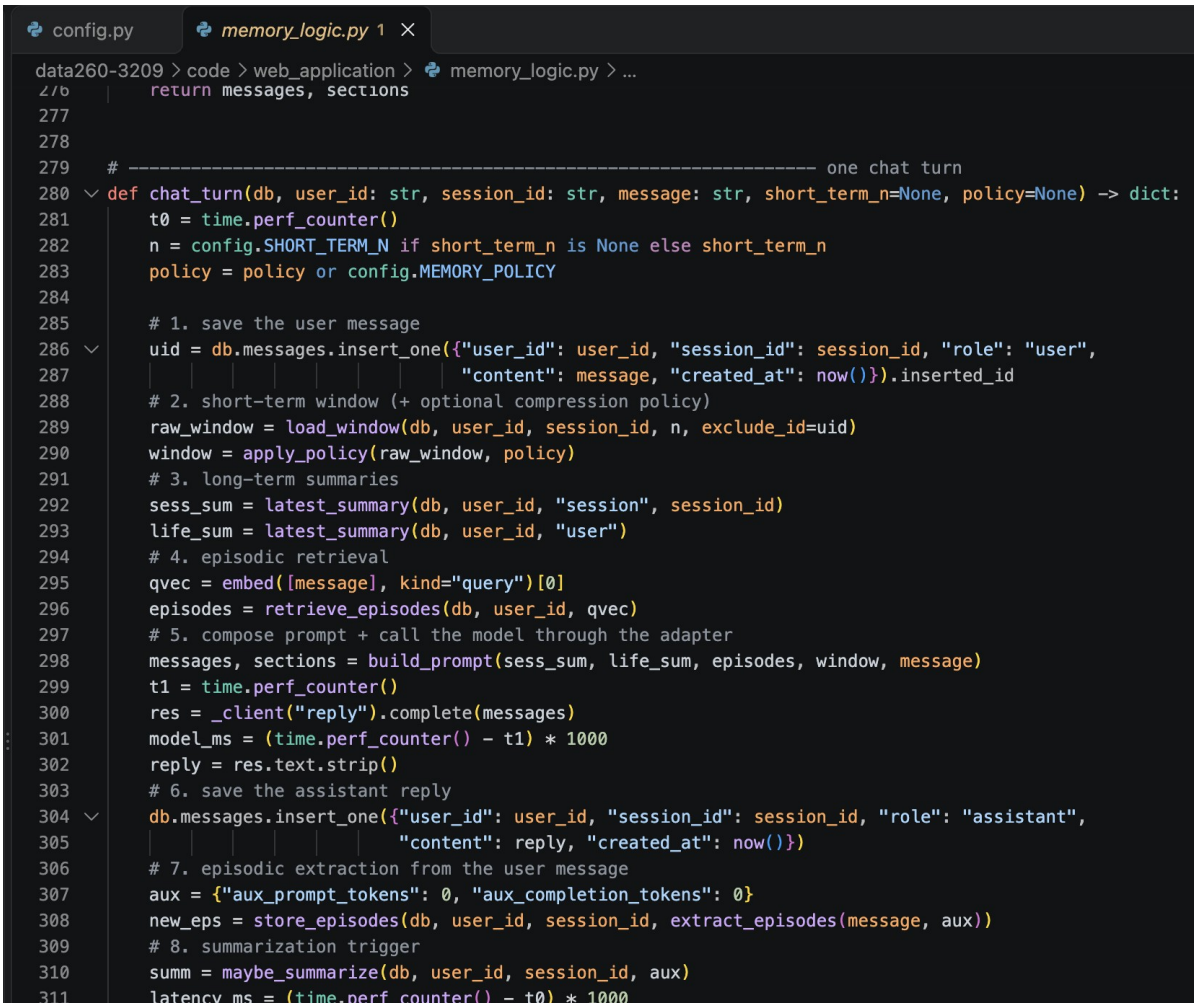
```python
@router.post("/chat", response_model=ChatOut)
def chat(body: ChatIn):
    session_id = body.session_id or f"sess-{uuid.uuid4().hex[:8]}"
    return mem.chat_turn(get_db(), body.user_id, session_id, body.message,
                         short_term_n=body.short_term_n, policy=body.policy)


@router.get("/memory/{user_id}")
def memory(user_id: str):
    db = get_db()
    msgs = list(db.messages.find({"user_id": user_id}, {"_id": 0, "user_id": 0})
                .sort([("created_at", -1), ("_id", -1)]).limit(16))
    sess = db.summaries.find_one({"user_id": user_id, "scope": "session"}, {"_id": 0},
                                 sort=[("created_at", -1)])
    life = db.summaries.find_one({"user_id": user_id, "scope": "user"}, {"_id": 0},
                                 sort=[("created_at", -1)])
    eps = list(db.episodes.find({"user_id": user_id}, {"_id": 0, "embedding": 0, "user_id": 0})
               .sort("created_at", -1).limit(20))
    return {"user_id": user_id, "recent_messages": list(reversed(msgs)),
            "session_summary": sess, "lifetime_summary": life, "episodic_facts": eps}


@router.get("/aggregate/{user_id}")
def aggregate(user_id: str):
    db = get_db()
    daily = list(db.messages.aggregate([
        {"$match": {"user_id": user_id}},
        {"$group": {"_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}},
                    "messages": {"$sum": 1},
                    "user_messages": {"$sum": {"$cond": [{"$eq": ["$role", "user"]}, 1, 0]}}}},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "date": "$_id", "messages": 1, "user_messages": 1}},
    ]))
    recent = list(db.summaries.find({"user_id": user_id}, {"_id": 0, "user_id": 0})
                  .sort("created_at", -1).limit(5))
    return {"user_id": user_id, "daily_message_counts": daily, "recent_summaries": recent}
```

Code 3 - One chat turn (memory_logic.py)

The eight server steps: save message, window, summaries, episodic retrieval, prompt + model call, save reply, fact extraction, summary trigger.



III. Memory Logic

Code 4 - Short-term window

Last N messages of the session, oldest first; the current message is excluded and added at the end.

```python
def load_window(db, user_id: str, session_id: str, n: int, exclude_id=None) -> list:
    """Sliding window: last n messages of this session (oldest first)."""
    if n <= 0:
        return []
    q = {"user_id": user_id, "session_id": session_id}
    if exclude_id is not None:
        q["_id"] = {"$ne": exclude_id}
    docs = list(db.messages.find(q).sort([("created_at", -1), ("_id", -1)]).limit(n))
    return [{"role": d["role"], "content": d["content"]} for d in reversed(docs)]
```

Code 5 - Episode extraction and storage

Up to 3 user-stated facts with importance; questions skipped; each fact embedded and stored, near-duplicates (cosine >= 0.95) skipped.

```python
# ---------------------------------------------------------------------- episodic extraction
EXTRACT_PROMPT = (
    "You store long-term memories. Extract up to 3 facts that the USER STATES about themselves, their "
    "trial or their work (names, IDs, numbers, dates, places, decisions). Copy the user's details "
    "exactly; never add advice, general knowledge or facts the user did not say. Questions, greetings "
    "and requests contain no facts. "
    'Return JSON only: {"facts": [{"fact": "<max 20 words>", "importance": <0..1>}]}. '
    'Examples: "Our site is in Fresno." -> {"facts": [{"fact": "Main site is in Fresno", "importance": 0.8}]}; '
    '"How do I report an adverse event?" -> {"facts": []}'
)
META_FACT = re.compile(r"^(you|the user) (asked|requested|want(ed)?|mentioned|said)\b", re.I)


def extract_episodes(message: str, aux=None) -> list:
    if message.strip().endswith("?"):          # pure questions carry no new facts
        return []
    res = _client("json").complete([{"role": "system", "content": EXTRACT_PROMPT},
                                    {"role": "user", "content": message}])
    _add_aux(aux, res)
    try:
        facts = json.loads(res.text).get("facts", [])
    except (json.JSONDecodeError, AttributeError):
        return []
    out = []
    for f in facts[:config.MAX_FACTS_PER_MESSAGE]:
        if isinstance(f, dict) and str(f.get("fact", "")).strip() and not META_FACT.match(str(f["fact"]).strip()):
            imp = f.get("importance", 0.5)
            imp = min(1.0, max(0.0, float(imp))) if isinstance(imp, (int, float)) else 0.5
            out.append({"fact": str(f["fact"]).strip()[:300], "importance": imp})
    return out


def store_episodes(db, user_id: str, session_id: str, facts: list) -> list:
    """Embed each fact and store it; skip near-duplicates (cosine >= 0.95)."""
    if not facts:
        return []
    vecs = embed([f["fact"] for f in facts])
    existing = [e["embedding"] for e in db.episodes.find({"user_id": user_id}, {"embedding": 1})]
    saved = []
    for f, v in zip(facts, vecs):
        if any(cosine(v, e) >= 0.95 for e in existing):
            continue
        db.episodes.insert_one({"user_id": user_id, "session_id": session_id, "fact": f["fact"],
                                "importance": float(f["importance"]), "embedding": v,
                                "created_at": now()})
        existing.append(v)
        saved.append(f)
    return saved

```

Code 6 - Embedding and retrieval

Ollama /api/embed with query/document prefixes; top-k facts by cosine similarity.

```python
def embed(texts: list, kind: str = "document") -> list:
    """kind='query' for the current message, 'document' for stored facts (nomic task prefixes)."""
    if not texts:
        return []
    prefix = ("search_query: " if kind == "query" else "search_document: ") \
        if "nomic" in config.EMBED_MODEL else ""
    try:
        r = requests.post(f"{config.OLLAMA_URL}/api/embed",
                          json={"model": config.EMBED_MODEL, "input": [prefix + t for t in texts]},
                          timeout=60)
        r.raise_for_status()
        _embed_backend["name"] = config.EMBED_MODEL
        return [[float(x) for x in v] for v in r.json()["embeddings"]]
    except Exception:
        _embed_backend["name"] = "hash-256 (fallback)"
        return [_hash_embed(t) for t in texts]

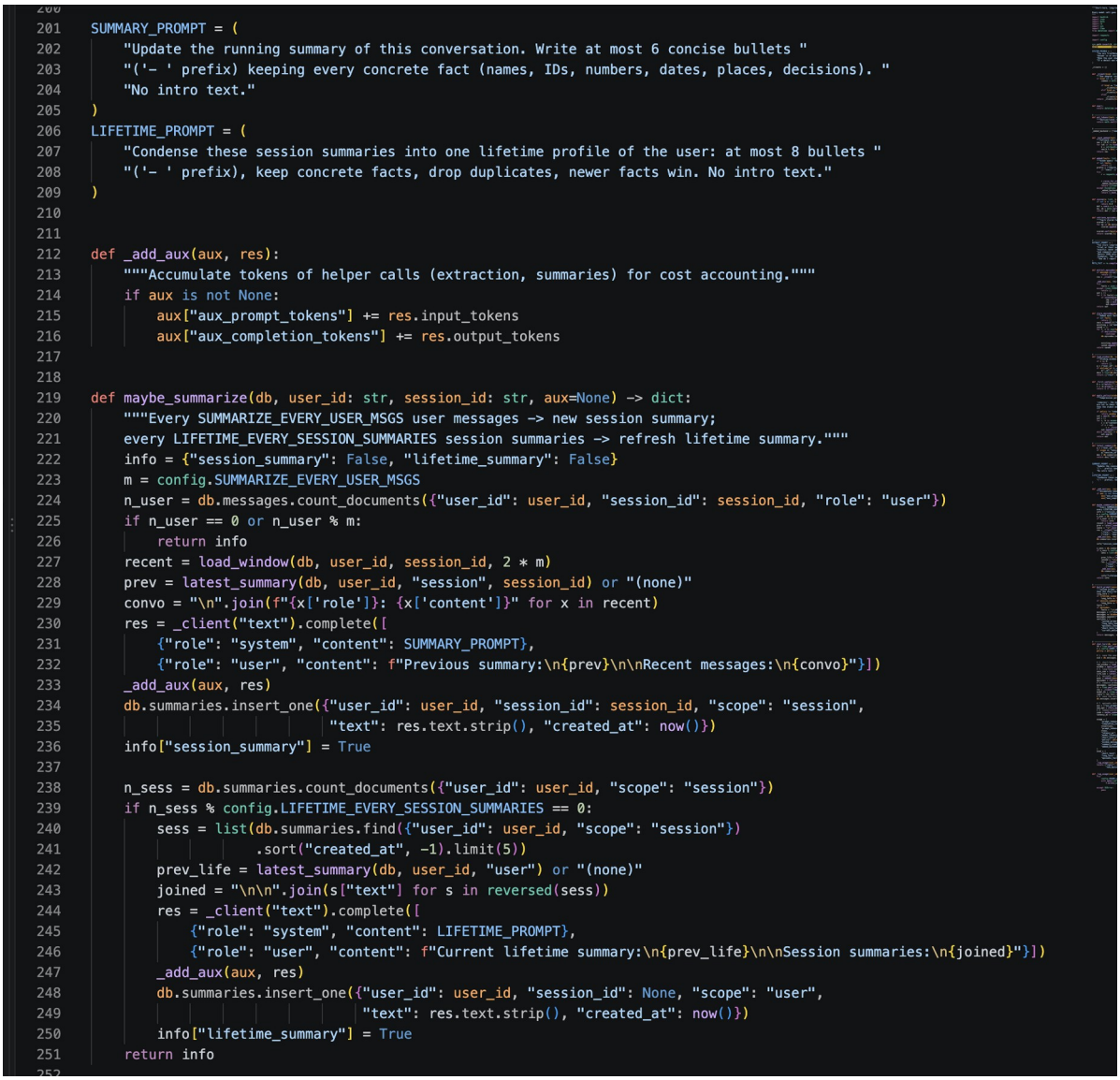

def cosine(a: list, b: list) -> float:
    if not a or not b or len(a) != len(b):
        return 0.0
    dot = sum(x * y for x, y in zip(a, b))
    na, nb = math.sqrt(sum(x * x for x in a)), math.sqrt(sum(y * y for y in b))
    return dot / (na * nb) if na and nb else 0.0


def retrieve_episodes(db, user_id: str, query_vec: list, k: int = config.EPISODES_TOP_K) -> list:
    """Top-k stored facts for this user by cosine similarity to the current message."""
    scored = []
    for ep in db.episodes.find({"user_id": user_id}, {"fact": 1, "importance": 1, "embedding": 1}):
        scored.append({"fact": ep["fact"], "importance": ep["importance"],
                       "score": round(cosine(query_vec, ep["embedding"]), 4)})
    scored.sort(key=lambda e: e["score"], reverse=True)
    return scored[:k]

```

Code 7 - Summarization trigger

Session summary every 5 user messages; lifetime summary (session_id null) every 2 session summaries.



```python
SUMMARY_PROMPT = (
    "Update the running summary of this conversation. Write at most 6 concise bullets "
    "('- ' prefix) keeping every concrete fact (names, IDs, numbers, dates, places, decisions). "
    "No intro text."
)
LIFETIME_PROMPT = (
    "Condense these session summaries into one lifetime profile of the user: at most 8 bullets "
    "('- ' prefix), keep concrete facts, drop duplicates, newer facts win. No intro text."
)


def _add_aux(aux, res):
    """Accumulate tokens of helper calls (extraction, summaries) for cost accounting."""
    if aux is not None:
        aux["aux_prompt_tokens"] += res.input_tokens
        aux["aux_completion_tokens"] += res.output_tokens


def maybe_summarize(db, user_id: str, session_id: str, aux=None) -> dict:
    """Every SUMMARIZE_EVERY_USER_MSGS user messages -> new session summary;
    every LIFETIME_EVERY_SESSION_SUMMARIES session summaries -> refresh lifetime summary."""
    info = {"session_summary": False, "lifetime_summary": False}
    m = config.SUMMARIZE_EVERY_USER_MSGS
    n_user = db.messages.count_documents({"user_id": user_id, "session_id": session_id, "role": "user"})
    if n_user == 0 or n_user % m:
        return info
    recent = load_window(db, user_id, session_id, 2 * m)
    prev = latest_summary(db, user_id, "session", session_id) or "(none)"
    convo = "\n".join(f"{x['role']}: {x['content']}" for x in recent)
    res = _client("text").complete([
        {"role": "system", "content": SUMMARY_PROMPT},
        {"role": "user", "content": f"Previous summary:\n{prev}\n\nRecent messages:\n{convo}"}])
    _add_aux(aux, res)
    db.summaries.insert_one({"user_id": user_id, "session_id": session_id, "scope": "session",
                             "text": res.text.strip(), "created_at": now()})
    info["session_summary"] = True

    n_sess = db.summaries.count_documents({"user_id": user_id, "scope": "session"})
    if n_sess % config.LIFETIME_EVERY_SESSION_SUMMARIES == 0:
        sess = list(db.summaries.find({"user_id": user_id, "scope": "session"})
                    .sort("created_at", -1).limit(5))
        prev_life = latest_summary(db, user_id, "user") or "(none)"
        joined = "\n\n".join(s["text"] for s in reversed(sess))
        res = _client("text").complete([
            {"role": "system", "content": LIFETIME_PROMPT},
            {"role": "user", "content": f"Current lifetime summary:\n{prev_life}\n\nSession summaries:\n{joined}"}])
        _add_aux(aux, res)
        db.summaries.insert_one({"user_id": user_id, "session_id": None, "scope": "user",
                                 "text": res.text.strip(), "created_at": now()})
        info["lifetime_summary"] = True
    return info
```

Code 8 - Prompt assembly

System primer + lifetime and session summaries + episodic facts, then the window, then the current message; token estimate per section.

```python
# ── prompt assembly ──────────────────────────────────────────────
def build_prompt(session_summary, lifetime_summary, episodes, window, message):
    """system primer + long-term summaries + episodic facts (system message),
    then the short-term window, then the current user message."""
    long_term = ""
    if lifetime_summary:
        long_term += f"\n\nLifetime summary of this user:\n{lifetime_summary}"
    if session_summary:
        long_term += f"\n\nSummary of this session so far:\n{session_summary}"
    facts = ""
    if episodes:
        facts = "\n\nRemembered facts (most relevant first):\n" + "\n".join(f"- {e['fact']}" for e in episodes)
    messages = [{"role": "system", "content": SYSTEM_PRIMER + long_term + facts}]
    messages += window
    messages.append({"role": "user", "content": message})
    sections = {
        "system_primer_tokens": est_tokens(SYSTEM_PRIMER),
        "long_term_tokens": est_tokens(long_term),
        "episodic_tokens": est_tokens(facts),
        "short_term_tokens": sum(est_tokens(m["content"]) for m in window),
        "current_message_tokens": est_tokens(message),
    }
    return messages, sections
```

IV. Run and Test

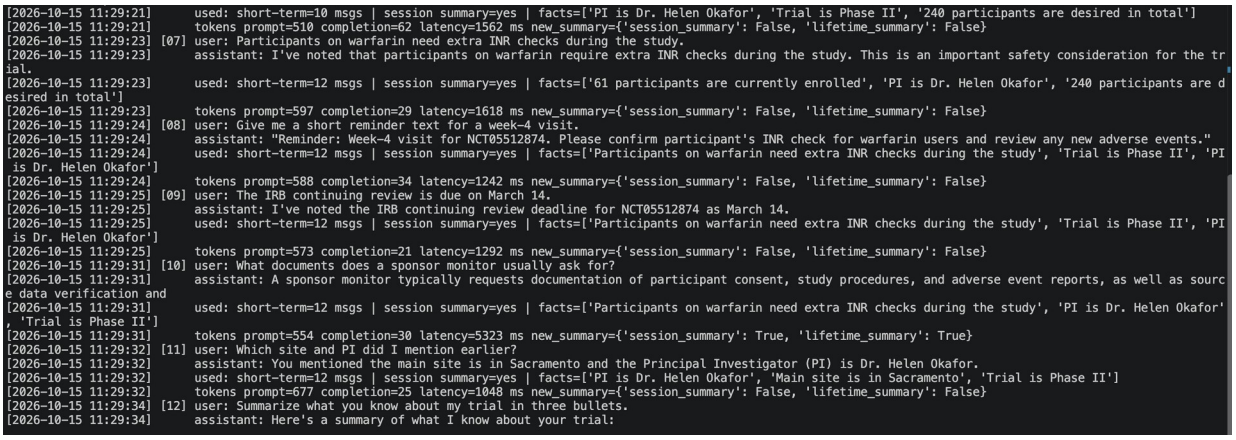
Output 4 - 12-message session, messages 1-6 (make hw06-part2)

Window grows by 2 each turn; facts appear from message 3; session summary created at message 5.

```
python3 code/run_hw06_part2.py
[2026-10-15 11:29:09] ======================================================================
[2026-10-15 11:29:09] RUN: python3 code/run_hw06_part2.py   (HW6 Part 2, user=ani_demo)   run by: Ani Taraiya
[2026-10-15 11:29:09] reset demo user ani_demo: removed 36 old docs
[2026-10-15 11:29:11] [01] user: Hi, I'm Ani, a clinical trial coordinator. I need help tracking my study.
[2026-10-15 11:29:11]      assistant: Nice to meet you, Ani. I'm TrialMate, your assistant. I'll help you track your study and stay organized. What's the name of your study?
[2026-10-15 11:29:11]      used: short-term=0 msgs | session summary=no | facts=[]
[2026-10-15 11:29:11]      tokens prompt=99 completion=36 latency=2202 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:13] [02] user: My study is NCT05512874, a Phase II trial of an oral GLP-1 drug for type 2 diabetes.
[2026-10-15 11:29:13]      assistant: I've noted the study ID and details. I'll keep track of NCT05512874. What's the next task or question you have regarding the trial?
[2026-10-15 11:29:13]      used: short-term=2 msgs | session summary=no | facts=[]
[2026-10-15 11:29:13]      tokens prompt=171 completion=34 latency=2417 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:15] [03] user: We want 240 participants in total and 61 are enrolled so far.
[2026-10-15 11:29:15]      assistant: I've noted the enrollment status. You have 61 participants enrolled out of a target of 240. What's the next update or task you'd like to
 discuss?
[2026-10-15 11:29:15]      used: short-term=4 msgs | session summary=no | facts=['Trial is Phase II', 'Study ID is NCT05512874', 'Drug is oral GLP-1']
[2026-10-15 11:29:15]      tokens prompt=262 completion=35 latency=1777 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:16] [04] user: What should I check before a screening visit?
[2026-10-15 11:29:16]      assistant: For a screening visit in a Phase II trial of an oral GLP-1 drug, I recommend verifying the participant's eligibility, reviewing their me
dical history, and confi
[2026-10-15 11:29:16]      used: short-term=6 msgs | session summary=no | facts=['Trial is Phase II', 'Drug is oral GLP-1', 'Study ID is NCT05512874']
[2026-10-15 11:29:16]      tokens prompt=315 completion=38 latency=924 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:20] [05] user: Our main site is in Sacramento and the PI is Dr. Helen Okafor.
[2026-10-15 11:29:20]      assistant: I've noted the main site location and the Principal Investigator's (PI) name. I'll keep this information in mind for future reference.
[2026-10-15 11:29:20]      used: short-term=8 msgs | session summary=no | facts=['Trial is Phase II', '61 participants are currently enrolled', 'Drug is oral GLP-1']
[2026-10-15 11:29:20]      tokens prompt=377 completion=30 latency=3591 ms new_summary={'session_summary': True, 'lifetime_summary': False}
[2026-10-15 11:29:21] [06] user: How do I report a serious adverse event?
[2026-10-15 11:29:21]      assistant: For a serious adverse event (SAE) in a clinical trial, you should immediately notify the Institutional Review Board (IRB), the sponsor,
and the Principal Invest
```

Output 5 - 12-message session, messages 7-12

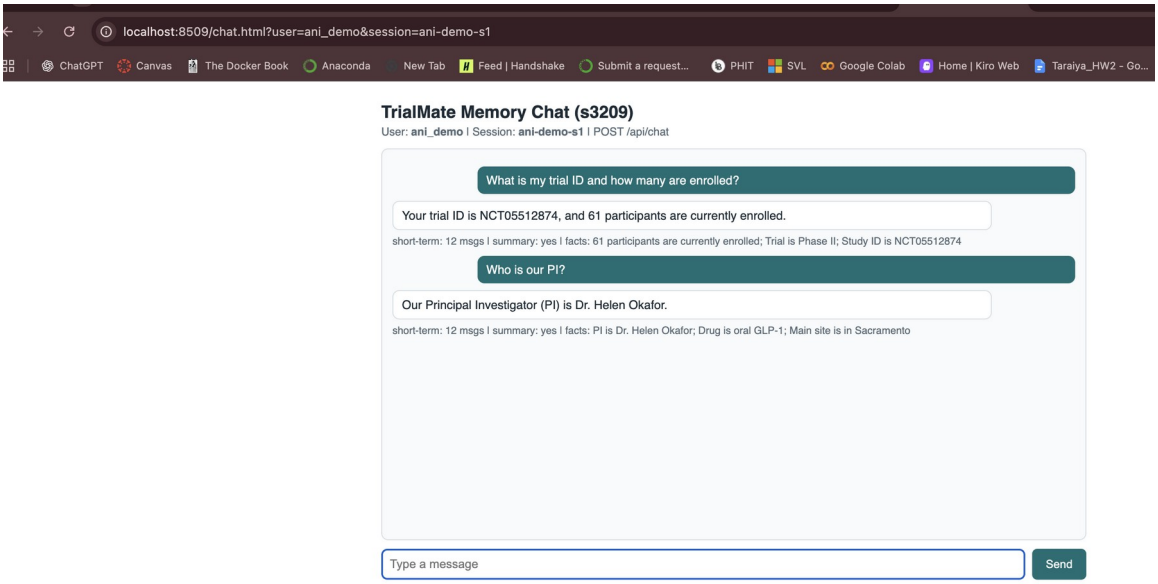
Window capped at 12; session and lifetime summaries at message 10; message 11 recalls site and PI.



```
[2026-10-15 11:29:21]      used: short-term=10 msgs | session summary=yes | facts=['PI is Dr. Helen Okafor', 'Trial is Phase II', '240 participants are desired in total']
[2026-10-15 11:29:21]      tokens prompt=510 completion=62 latency=1562 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:23] [07] user: Participants on warfarin need extra INR checks during the study.
[2026-10-15 11:29:23]      assistant: I've noted that participants on warfarin require extra INR checks during the study. This is an important safety consideration for the tr
ial.
[2026-10-15 11:29:23]      used: short-term=12 msgs | session summary=yes | facts=['61 participants are currently enrolled', 'PI is Dr. Helen Okafor', '240 participants are d
esired in total']
[2026-10-15 11:29:23]      tokens prompt=597 completion=29 latency=1618 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:24] [08] user: Give me a short reminder text for a week-4 visit.
[2026-10-15 11:29:24]      assistant: "Reminder: Week-4 visit for NCT05512874. Please confirm participant's INR check for warfarin users and review any new adverse events."
[2026-10-15 11:29:24]      used: short-term=12 msgs | session summary=yes | facts=['Participants on warfarin need extra INR checks during the study', 'Trial is Phase II', 'PI
 is Dr. Helen Okafor']
[2026-10-15 11:29:24]      tokens prompt=588 completion=34 latency=1242 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:25] [09] user: The IRB continuing review is due on March 14.
[2026-10-15 11:29:25]      assistant: I've noted the IRB continuing review deadline for NCT05512874 as March 14.
[2026-10-15 11:29:25]      used: short-term=12 msgs | session summary=yes | facts=['Participants on warfarin need extra INR checks during the study', 'Trial is Phase II', 'PI
 is Dr. Helen Okafor']
[2026-10-15 11:29:25]      tokens prompt=573 completion=21 latency=1292 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:31] [10] user: What documents does a sponsor monitor usually ask for?
[2026-10-15 11:29:31]      assistant: A sponsor monitor typically requests documentation of participant consent, study procedures, and adverse event reports, as well as sourc
e data verification and
[2026-10-15 11:29:31]      used: short-term=12 msgs | session summary=yes | facts=['Participants on warfarin need extra INR checks during the study', 'PI is Dr. Helen Okafor'
, 'Trial is Phase II']
[2026-10-15 11:29:31]      tokens prompt=554 completion=30 latency=5323 ms new_summary={'session_summary': True, 'lifetime_summary': True}
[2026-10-15 11:29:32] [11] user: Which site and PI did I mention earlier?
[2026-10-15 11:29:32]      assistant: You mentioned the main site is in Sacramento and the Principal Investigator (PI) is Dr. Helen Okafor.
[2026-10-15 11:29:32]      used: short-term=12 msgs | session summary=yes | facts=['PI is Dr. Helen Okafor', 'Main site is in Sacramento', 'Trial is Phase II']
[2026-10-15 11:29:32]      tokens prompt=677 completion=25 latency=1048 ms new_summary={'session_summary': False, 'lifetime_summary': False}
[2026-10-15 11:29:34] [12] user: Summarize what you know about my trial in three bullets.
[2026-10-15 11:29:34]      assistant: Here's a summary of what I know about your trial:
```

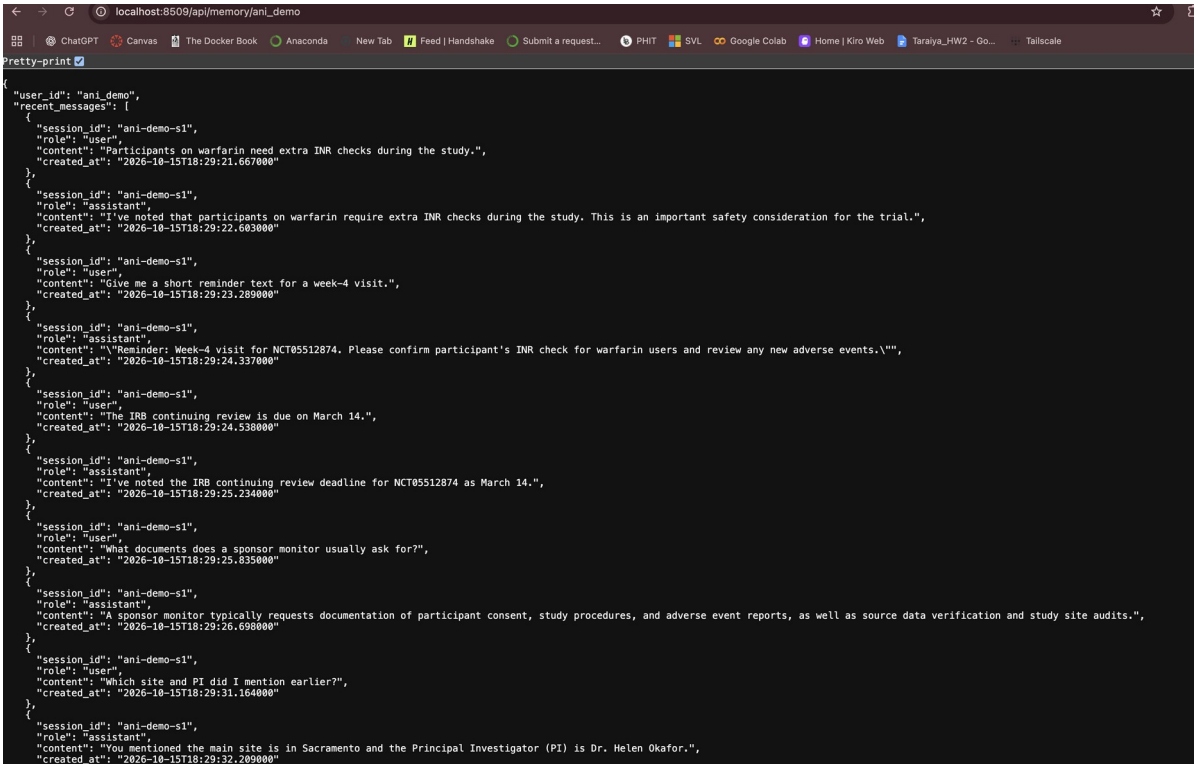
Output 6 - Conversation in the chat page

Two extra questions answered from memory; the grey line shows window size, summary use and facts retrieved.



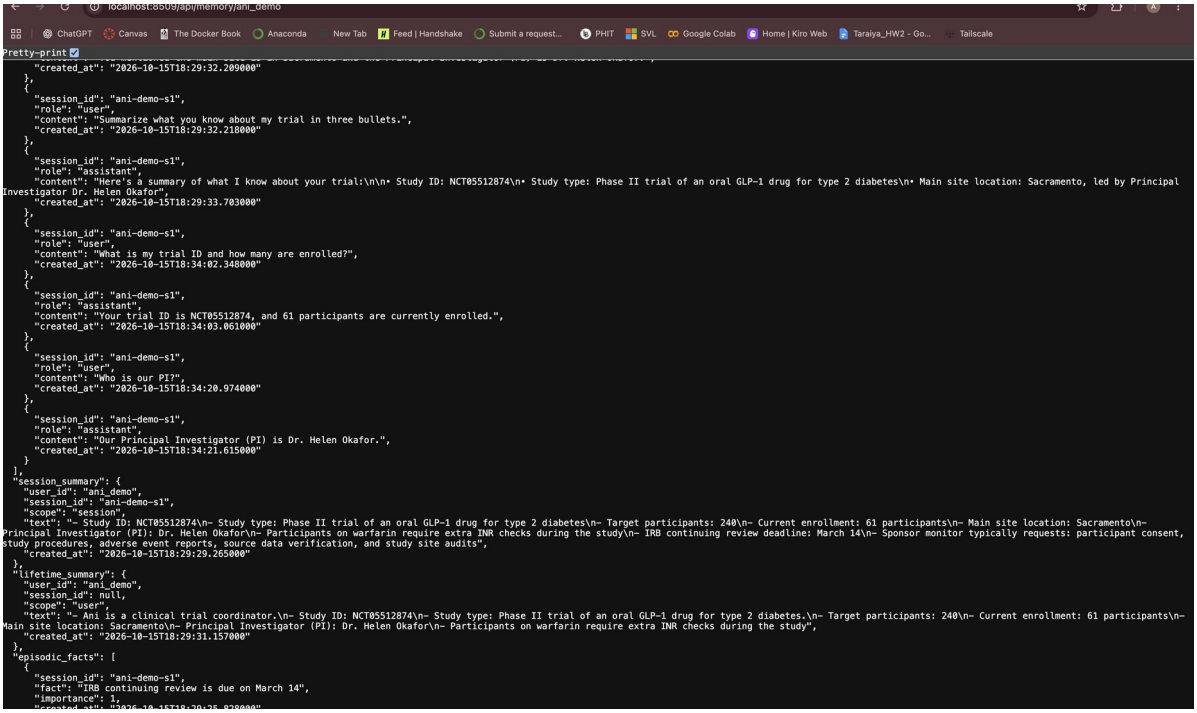
Output 7 - GET /api/memory/ani_demo (1 of 3)

Last 16 messages.



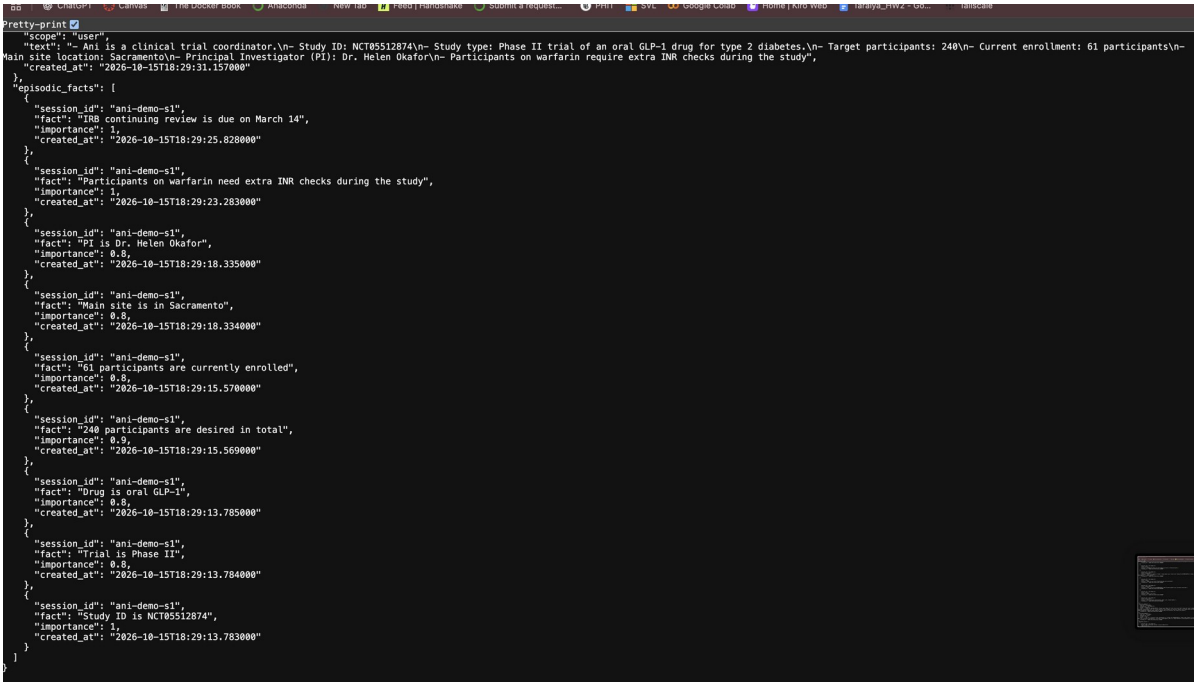
Output 7 - GET /api/memory/ani_demo (2 of 3)

End of messages, latest session summary and lifetime summary.



Output 7 - GET /api/memory/ani_demo (3 of 3)

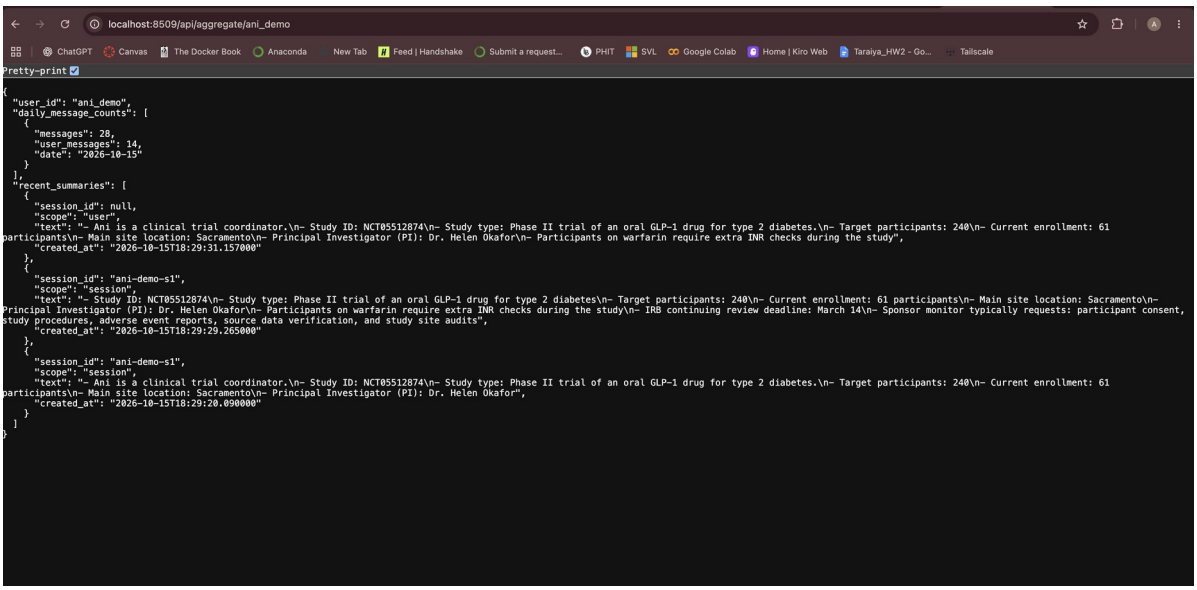
Episodic facts (embedding hidden in the response).



```
Pretty-print
  "scope": "user",
  "text": "- Ani is a clinical trial coordinator.\n- Study ID: NCT05512874\n- Study type: Phase II trial of an oral GLP-1 drug for type 2 diabetes.\n- Target participants: 240\n- Current enrollment: 61 participants\n-
Main site location: Sacramento\n- Principal Investigator (PI): Dr. Helen Okafor\n- Participants on warfarin require extra INR checks during the study",
  "created_at": "2026-10-15T18:29:31.157000"
 },
 "episodic_facts": [
  {
   "session_id": "ani-demo-s1",
   "fact": "IRB continuing review is due on March 14",
   "importance": 1,
   "created_at": "2026-10-15T18:29:25.828000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "Participants on warfarin need extra INR checks during the study",
   "importance": 1,
   "created_at": "2026-10-15T18:29:23.283000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "PI is Dr. Helen Okafor",
   "importance": 0.8,
   "created_at": "2026-10-15T18:29:18.335000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "Main site is in Sacramento",
   "importance": 0.8,
   "created_at": "2026-10-15T18:29:18.334000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "61 participants are currently enrolled",
   "importance": 0.8,
   "created_at": "2026-10-15T18:29:15.570000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "240 participants are desired in total",
   "importance": 0.9,
   "created_at": "2026-10-15T18:29:15.569000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "Drug is oral GLP-1",
   "importance": 0.8,
   "created_at": "2026-10-15T18:29:13.785000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "Trial is Phase II",
   "importance": 0.8,
   "created_at": "2026-10-15T18:29:13.784000"
  },
  {
   "session_id": "ani-demo-s1",
   "fact": "Study ID is NCT05512874",
   "importance": 1,
   "created_at": "2026-10-15T18:29:13.783000"
  }
 ]
}
```


Output 8 - GET /api/aggregate/ani_demo

28 messages (14 user) on 2026-10-15 and the three most recent summaries.



```
localhost:8509/api/aggregate/ani_demo
Pretty-print
{
 "user_id": "ani_demo",
 "daily_message_counts": [
  {
   "messages": 28,
   "user_messages": 14,
   "date": "2026-10-15"
  }
 ],
 "recent_summaries": [
  {
   "session_id": null,
   "scope": "user",
   "text": "- Ani is a clinical trial coordinator.\n- Study ID: NCT05512874\n- Study type: Phase II trial of an oral GLP-1 drug for type 2 diabetes.\n- Target participants: 240\n- Current enrollment: 61
participants\n- Main site location: Sacramento\n- Principal Investigator (PI): Dr. Helen Okafor\n- Participants on warfarin require extra INR checks during the study",
   "created_at": "2026-10-15T18:29:31.157000"
  },
  {
   "session_id": "ani-demo-s1",
   "scope": "session",
   "text": "- Study ID: NCT05512874\n- Study type: Phase II trial of an oral GLP-1 drug for type 2 diabetes\n- Target participants: 240\n- Current enrollment: 61 participants\n- Main site location: Sacramento\n-
Principal Investigator (PI): Dr. Helen Okafor\n- Participants on warfarin require extra INR checks during the study\n- IRB continuing review deadline: March 14\n- Sponsor monitor typically requests: participant consent,
study procedures, adverse event reports, source data verification, and study site audits",
   "created_at": "2026-10-15T18:29:29.265000"
  },
  {
   "session_id": "ani-demo-s1",
   "scope": "session",
   "text": "- Ani is a clinical trial coordinator.\n- Study ID: NCT05512874\n- Study type: Phase II trial of an oral GLP-1 drug for type 2 diabetes.\n- Target participants: 240\n- Current enrollment: 61
participants\n- Main site location: Sacramento\n- Principal Investigator (PI): Dr. Helen Okafor",
   "created_at": "2026-10-15T18:29:20.090000"
  }
 ]
}
```


- Short-term: window 0, 2, 4 ... 12 messages, then fixed at 12 (SHORT_TERM_N).
- Long-term: summaries created at user messages 5 and 10; lifetime summary refreshed at message 10.
- Episodic: 9 facts from 12 messages; questions add no facts; the right facts are retrieved for "Which site and PI did I mention earlier?" and "Who is our PI?".

Part 3 - What Memory Costs

Same 30-message conversation (reports/hw06/cases/conversation_30.yaml) in a fresh user and session for each setting, then the 5 probes. Probe facts are planted at messages 2, 8, 14, 20 and 26. Model llama3.1:8b, temperature 0, seed 3209.

1. Log Memory Usage

Code 1 - Per-request usage record (memory_logic.py)

Every /api/chat call returns and logs prompt/completion tokens, tokens per section, and latency.

```
    latency_ms = (time.perf_counter() - t0) * 1000

    usage = {
        "prompt_tokens": res.input_tokens,
        "completion_tokens": res.output_tokens,
        **sections,
        "prompt_tokens_est": sum(sections.values()),
        **aux,
        "latency_ms": round(latency_ms, 1),
        "model_latency_ms": round(model_ms, 1),
        "short_term_n": n,
        "policy": policy,
        "window_messages": len(window),
        "summary_created": summ,
        "embed_backend": _embed_backend["name"],
    }
    used = {
        "short_term": {"count": len(window), "messages": window},
        "long_term": {"session_summary": sess_sum, "lifetime_summary": life_sum},
        "episodic_facts": episodes,
    }
    _log_usage(user_id, session_id, message, usage)
    return {"reply": reply, "session_id": session_id, "used": used, "usage": usage,
            "new_episodes": new_eps}


def _log_usage(user_id, session_id, message, usage):
    try:
        config.USAGE_LOG.parent.mkdir(parents=True, exist_ok=True)
        with open(config.USAGE_LOG, "a") as f:
            f.write(json.dumps({"ts": now().isoformat(timespec="seconds"), "user_id": user_id,
                                "session_id": session_id, "message": message[:80], **usage}) + "\n")
    except OSError:
        pass
```

Raw file	Content
raw/chat_requests.jsonl	One JSON line per /api/chat request (216 requests)
raw/part3_turns.csv	140 rows: 4 settings x (30 turns + 5 probes)
raw/part3_probe_scores.json	Probe replies, criteria met and scores
raw/part3_summary.json	Means and costs per setting

Output 1 - Raw per-turn data (part3_turns.csv)

Prompt and completion tokens, section tokens (primer, long-term, short-term, episodic) and latency per turn.

config.py | part3_turns.csv U

data260-3209 > reports > hw06 > raw > part3_turns.csv

```
setting,phase,turn,prompt_tokens,completion_tokens,system_primer_tokens,long_term_tokens,short_term_tokens,episodic_tokens,current_message_tokens,prompt_tokens_es
n4,conversation,1.0,100.0,26.0,74.0,0.0,0.0,0.0,25.0,99.0,0.0,0.0,698.2,670.8,0.0,0.0,0.0,0.0
n4,conversation,2.0,166.0,44.0,74.0,0.0,54.0,0.0,30.0,158.0,191.0,59.0,2184.9,938.6,2.0,0.0,0.0,0.0,0.0
n4,conversation,3.0,279.0,50.0,74.0,0.0,127.0,37.0,23.0,261.0,0.0,0.0,1137.1,1124.6,4.0,0.0,0.0,2.0
n4,conversation,4.0,292.0,39.0,74.0,0.0,162.0,37.0,13.0,286.0,0.0,0.0,928.8,914.7,4.0,0.0,0.0,2.0
n4,conversation,5.0,267.0,53.0,74.0,0.0,158.0,37.0,14.0,283.0,386.0,93.0,3209.1,1168.8,4.0,1.0,0.0,2.0
n4,conversation,6.0,364.0,6.0,74.0,122.0,153.0,37.0,16.0,402.0,0.0,0.0,401.5,386.5,4.0,0.0,0.0,2.0
n4,conversation,7.0,332.0,50.0,74.0,122.0,104.0,37.0,13.0,350.0,170.0,6.0,1240.3,1033.6,4.0,0.0,0.0,2.0
n4,conversation,8.0,338.0,20.0,74.0,122.0,95.0,37.0,23.0,351.0,180.0,46.0,1561.2,550.2,4.0,0.0,0.0,2.0
n4,conversation,9.0,359.0,26.0,74.0,122.0,120.0,53.0,17.0,386.0,0.0,0.0,679.3,665.7,4.0,0.0,0.0,3.0
n4,conversation,10.0,334.0,27.0,74.0,122.0,100.0,53.0,13.0,362.0,697.0,279.0,6470.0,683.1,4.0,1.0,1.0,3.0
n4,conversation,11.0,527.0,6.0,74.0,357.0,107.0,53.0,19.0,610.0,0.0,0.0,575.2,559.4,4.0,0.0,0.0,3.0
n4,conversation,12.0,510.0,59.0,74.0,357.0,75.0,52.0,18.0,576.0,0.0,0.0,1232.2,1219.0,4.0,0.0,0.0,3.0
n4,conversation,13.0,542.0,51.0,74.0,357.0,107.0,52.0,11.0,601.0,0.0,0.0,1162.0,1146.7,4.0,0.0,0.0,3.0
n4,conversation,14.0,592.0,26.0,74.0,357.0,160.0,52.0,22.0,665.0,179.0,48.0,1692.2,683.0,4.0,0.0,0.0,3.0
n4,conversation,15.0,542.0,54.0,74.0,357.0,127.0,46.0,9.0,613.0,509.0,231.0,5902.6,1207.7,4.0,1.0,0.0,3.0
n4,conversation,16.0,613.0,57.0,74.0,430.0,127.0,52.0,11.0,694.0,0.0,0.0,1300.8,1284.6,4.0,0.0,0.0,3.0
```

Prompt and completion tokens are Ollama counts; section tokens are estimates (characters / 4).

2. Memory Probes

Code 2 - memory_probes.yaml

Five fixed questions, keyword criteria and the 1-5 rubric, written before the runs.

config.py | memory_probes.yaml

data260-3209 > reports > hw06 > memory_probes.yaml

```yaml
# HW6 Part 3 — memory probes (written before any test run)
# Each probe is asked after the 30-message conversation, in the same session.
# Scoring is automatic and objective: case-insensitive keyword groups.
#   A criterion is met when ANY of its keywords appears in the reply (no letter/digit right
#   before it and no digit right after it, so "March 1" does not match "March 14").

rubric:
  "5": "All criteria met — correct, complete recall"
  "4": "Most criteria met (>= 75%)"
  "3": "Half of the criteria met — partial recall"
  "2": "No criterion met, but the model honestly says it does not know (no invented value)"
  "1": "No criterion met and the reply invents or states a wrong value"
scoring: "score = 1 + round(4 * met / total); if met == 0 and the reply abstains -> 2"
abstain_keywords: ["don't know", "do not know", "not mentioned", "didn't mention", "did not mention", "no information", "not sure", "haven't mentioned", "have not

probes:
  - id: P1
    question: "What is the registration number of the trial I am coordinating?"
    planted_at_message: 2
    criteria:
      - ["NCT05512874", "05512874"]
  - id: P2
    question: "What is our enrollment target, and how many participants are enrolled so far?"
    planted_at_message: 8
    criteria:
      - ["240"]
      - ["61"]
  - id: P3
    question: "Where is our main site, and who is the site principal investigator?"
    planted_at_message: 14
    criteria:
      - ["Sacramento"]
      - ["Okafor"]
  - id: P4
    question: "Which medication did I flag as a safety concern, and what extra check is needed?"
    planted_at_message: 20
    criteria:
      - ["warfarin"]
      - ["INR"]
  - id: P5
    question: "When is our IRB continuing review due, and when must the renewal packet be ready?"
    planted_at_message: 26
    criteria:
      - ["March 14", "Mar 14", "14 March", "3/14"]
      - ["March 1", "Mar 1", "1 March", "3/1"]
```

3. Short-Term Memory Sizes

Code 3 - Settings and probe scoring (run_hw06_memory_cost.py)

Settings, prices, logged fields and the automatic 1-5 scorer.

```python
    "n4": (4, "none"),
    "n12": (12, "none"),
    "n30": (30, "none"),
    "compress": (30, "compress"),  # memory-saving policy applied to the N=30 window
}
LABELS = {"n4": "SHORT_TERM_N=4", "n12": "SHORT_TERM_N=12", "n30": "SHORT_TERM_N=30",
          "compress": "Compression policy (N=30 + compress)"}
PRICE_IN, PRICE_OUT = 1.0 / 1e6, 5.0 / 1e6     # $ per token
CONVERSATIONS = 10_000

TURN_FIELDS = ["setting", "phase", "turn", "prompt_tokens", "completion_tokens",
               "system_primer_tokens", "long_term_tokens", "short_term_tokens", "episodic_tokens",
               "current_message_tokens", "prompt_tokens_est", "aux_prompt_tokens",
               "aux_completion_tokens", "latency_ms", "model_latency_ms", "window_messages",
               "session_summary", "lifetime_summary", "episodes_retrieved"]


def log(msg=""):
    line = f"[{datetime.now():%Y-%m-%d %H:%M:%S}] {msg}"
    print(line, flush=True)
    with open(HW / "RUN_LOG.txt", "a") as f:
        f.write(line + "\n")


def score_reply(reply: str, probe: dict, abstain: list) -> tuple:
    text = reply.lower()
    met = 0
    for group in probe["criteria"]:
        if any(re.search(r"(?<![0-9a-z])" + re.escape(k.lower()) + r"(?!\d)", text) for k in group):
            met += 1
    total = len(probe["criteria"])
    if met == 0:
        return (2 if any(a in text for a in abstain) else 1), met, total
    return 1 + round(4 * met / total), met, total

```

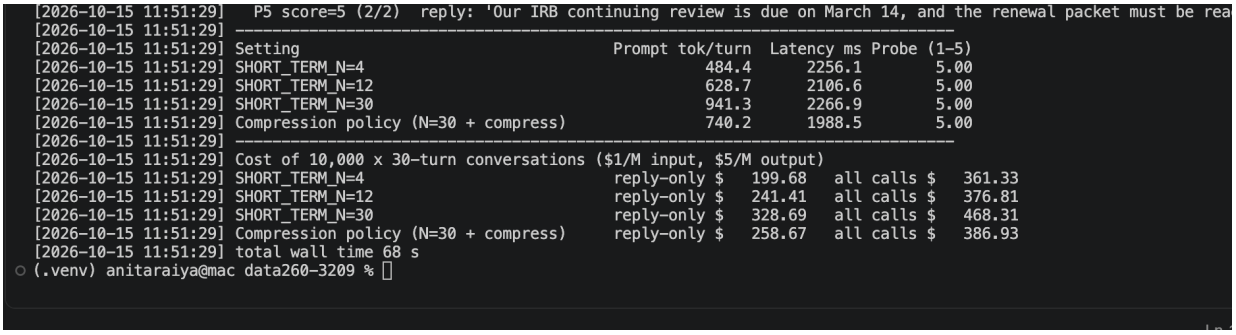
Output 2 - One run (compression setting)

30 turns with tokens, window size and latency, then the 5 probe scores.

```
(.venv) anitaraiya@mac data260-3209 % clear
python3 code/run_hw06_memory_cost.py --only compress
[2026-10-15 11:50:21] ========================================================================
[2026-10-15 11:50:21] RUN: python3 code/run_hw06_memory_cost.py   (HW6 Part 3)   run by: Ani Taraiya
[2026-10-15 11:50:21] backend http://localhost:8509  conversation=30 msgs  probes=5  stamp=10151150
[2026-10-15 11:50:21] --- Compression policy (N=30 + compress)  user_id=cost_compress_10151150  policy=compress
[2026-10-15 11:50:23]   turn 01  prompt=  100  completion=  26  window= 0  latency=    1413 ms
[2026-10-15 11:50:25]   turn 02  prompt=  166  completion=  44  window= 2  latency=    2249 ms
[2026-10-15 11:50:26]   turn 03  prompt=  279  completion=  50  window= 4  latency=    1129 ms
[2026-10-15 11:50:27]   turn 04  prompt=  332  completion=  29  window= 6  latency=     747 ms
[2026-10-15 11:50:30]   turn 05  prompt=  361  completion=  48  window= 8  latency=    2878 ms
[2026-10-15 11:50:31]   turn 06  prompt=  493  completion=  13  window=10  latency=     641 ms
[2026-10-15 11:50:31]   turn 07  prompt=  520  completion=  20  window=12  latency=     711 ms
[2026-10-15 11:50:33]   turn 08  prompt=  545  completion=  13  window=14  latency=    1523 ms
[2026-10-15 11:50:33]   turn 09  prompt=  588  completion=  13  window=16  latency=     667 ms
[2026-10-15 11:50:39]   turn 10  prompt=  619  completion=  23  window=18  latency=    5274 ms
[2026-10-15 11:50:40]   turn 11  prompt=  796  completion=  13  window=20  latency=     940 ms
[2026-10-15 11:50:41]   turn 12  prompt=  833  completion=  21  window=22  latency=     870 ms
[2026-10-15 11:50:42]   turn 13  prompt=  872  completion=  44  window=24  latency=    1348 ms
[2026-10-15 11:50:44]   turn 14  prompt=  868  completion=  11  window=23  latency=    1750 ms
[2026-10-15 11:50:47]   turn 15  prompt=  856  completion=  13  window=24  latency=    3453 ms
[2026-10-15 11:50:48]   turn 16  prompt=  897  completion=  26  window=26  latency=     988 ms
[2026-10-15 11:50:51]   turn 17  prompt=  889  completion=  34  window=26  latency=    2442 ms
[2026-10-15 11:50:52]   turn 18  prompt=  882  completion=  26  window=25  latency=    1001 ms
[2026-10-15 11:50:53]   turn 19  prompt=  903  completion=  38  window=26  latency=    1250 ms
[2026-10-15 11:51:02]   turn 20  prompt=  889  completion=  19  window=24  latency=    9380 ms
[2026-10-15 11:51:04]   turn 21  prompt= 1014  completion=  36  window=24  latency=    1479 ms
[2026-10-15 11:51:04]   turn 22  prompt= 1007  completion=  13  window=23  latency=     737 ms
[2026-10-15 11:51:05]   turn 23  prompt= 1005  completion=  13  window=24  latency=     748 ms
[2026-10-15 11:51:07]   turn 24  prompt= 1021  completion=  30  window=26  latency=    1732 ms
[2026-10-15 11:51:09]   turn 25  prompt= 1021  completion=  13  window=25  latency=    2320 ms
[2026-10-15 11:51:11]   turn 26  prompt=  887  completion=  13  window=26  latency=    1769 ms
[2026-10-15 11:51:12]   turn 27  prompt=  880  completion=  13  window=26  latency=     729 ms
[2026-10-15 11:51:12]   turn 28  prompt=  880  completion=  13  window=27  latency=     726 ms
[2026-10-15 11:51:13]   turn 29  prompt=  896  completion=  13  window=28  latency=     731 ms
[2026-10-15 11:51:21]   turn 30  prompt=  908  completion=  51  window=28  latency=    8028 ms
[2026-10-15 11:51:22]   P1 score=5 (1/1)  reply: 'The registration number of the trial is NCT05512874.'
[2026-10-15 11:51:23]   P2 score=5 (2/2)  reply: 'Our enrollment target is 240 participants across all sites, and 61 participants are currently enrolled.'
[2026-10-15 11:51:24]   P3 score=5 (2/2)  reply: 'Our main site is located in Sacramento, and the site principal investigator is Dr. Helen Okafor.'
[2026-10-15 11:51:25]   P4 score=5 (2/2)  reply: 'You flagged warfarin as a safety concern, and participants taking this medication need extra INR checks due to'
[2026-10-15 11:51:29]   P5 score=5 (2/2)  reply: 'Our IRB continuing review is due on March 14, and the renewal packet must be ready by March 1.'
[2026-10-15 11:51:29] ------------------------------------------------------------------------
```

Output 3 - Results for all settings

Mean prompt tokens per turn, mean latency, probe quality, and cost of 10,000 conversations.



Memory-cost table

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	484.4	2256.1	5.00
SHORT_TERM_N=12	628.7	2106.6	5.00
SHORT_TERM_N=30	941.3	2266.9	5.00
Compression policy (N=30 + compress)	740.2	1988.5	5.00

Where the prompt tokens go (mean per turn)

Setting	Short-term	Long-term	Episodic	Model latency (ms)
SHORT_TERM_N=4	118.4	285.0	47.6	877.8
SHORT_TERM_N=12	328.2	211.8	47.6	996.0
SHORT_TERM_N=30	612.4	249.2	47.6	1136.4
Compression policy	394.5	219.0	47.6	901.9

- Prompt size grows with N: +30% from 4 to 12 and +94% from 4 to 30; the short-term window drives it.
- Model time grows with the prompt (878 to 1136 ms). Request latency stays near 2.1-2.3 s because fact extraction and summary calls add the same time to every setting.
- All settings scored 5/5 on every probe: facts outside the window were still recalled from the summaries and the episodic facts, so with this memory design a small window loses no recall.

4. Memory-Saving Policy

Code 4 - Compression policy (memory_logic.py)

Newest 4 messages verbatim; older assistant replies cut to the first sentence, older user messages to 200 characters; oldest evicted above a 500-token budget.

```python
def _first_sentence(text: str, limit: int = 120) -> str:
    m = re.match(r"(.+?[.!?])(\s|$)", text.strip(), flags=re.S)
    s = (m.group(1) if m else text.strip())
    return s if len(s) <= limit else s[:limit].rstrip() + "..."

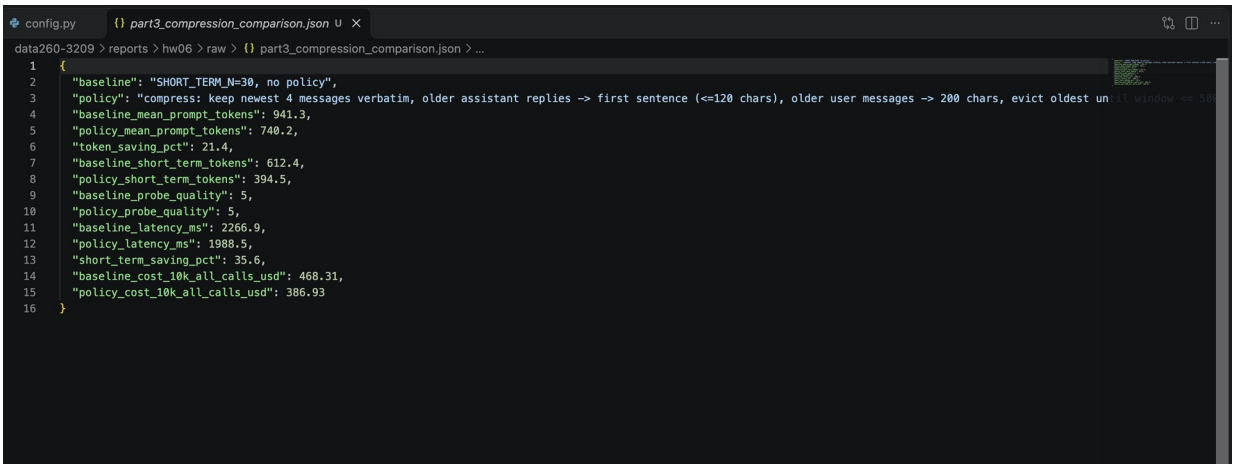

def apply_policy(window: list, policy: str, keep_recent: int = 4, budget: int = 500) -> list:
    """Compression policy (Part 3.4).

    'compress': the newest `keep_recent` messages stay verbatim; older assistant replies
    are cut to their first sentence (<=120 chars) and older user messages to 200 chars;
    then the oldest messages are evicted until the window fits `budget` estimated tokens.
    """
    if policy != "compress" or not window:
        return window
    cut = max(0, len(window) - keep_recent)
    out = []
    for i, m in enumerate(window):
        c = m["content"]
        if i < cut:
            c = _first_sentence(c) if m["role"] == "assistant" else (c[:200] + ("..." if len(c) > 200 else ""))
        out.append({"role": m["role"], "content": c})
    while len(out) > keep_recent and sum(est_tokens(m["content"]) for m in out) > budget:
        out.pop(0)
    return out

```

Output 4 - Compression comparison (part3_compression_comparison.json)

Policy applied to the N=30 window and compared with plain N=30.



Metric	N=30 (baseline)	N=30 + compress	Change
Mean prompt tokens/turn	941.3	740.2	-21.4%
Short-term tokens/turn	612.4	394.5	-35.6%
Probe quality (1-5)	5.00	5.00	no change
Mean latency (ms)	2266.9	1988.5	-12.3%
Cost of 10,000 conversations (all calls)	$468.31	$386.93	-17.4%

5. Memory Cost Estimate

Code 5 - Means and cost calculation (run_hw06_memory_cost.py)

Cost = 10,000 x (input tokens x $1/M + output tokens x $5/M) per 30-turn conversation.

```python
def summarize(rows, scores):
    out = {}
    for name in SETTINGS:
        conv = [r for r in rows if r["setting"] == name and r["phase"] == "conversation"]
        allr = [r for r in rows if r["setting"] == name]
        sc = [s["score"] for s in scores if s["setting"] == name]
        if not conv:
            continue
        p_in = sum(r["prompt_tokens"] + r["aux_prompt_tokens"] for r in conv)
        p_out = sum(r["completion_tokens"] + r["aux_completion_tokens"] for r in conv)
        reply_in = sum(r["prompt_tokens"] for r in conv)
        reply_out = sum(r["completion_tokens"] for r in conv)
        out[name] = {
            "label": LABELS[name],
            "mean_prompt_tokens_per_turn": round(statistics.mean(r["prompt_tokens"] for r in conv), 1),
            "mean_prompt_tokens_est_per_turn": round(statistics.mean(r["prompt_tokens_est"] for r in conv), 1),
            "mean_short_term_tokens": round(statistics.mean(r["short_term_tokens"] for r in conv), 1),
            "mean_long_term_tokens": round(statistics.mean(r["long_term_tokens"] for r in conv), 1),
            "mean_episodic_tokens": round(statistics.mean(r["episodic_tokens"] for r in conv), 1),
            "mean_completion_tokens": round(statistics.mean(r["completion_tokens"] for r in conv), 1),
            "mean_latency_ms": round(statistics.mean(r["latency_ms"] for r in conv), 1),
            "mean_model_latency_ms": round(statistics.mean(r["model_latency_ms"] for r in conv), 1),
            "probe_quality": round(statistics.mean(sc), 2) if sc else None,
            "probe_scores": sc,
            "tokens_per_conversation": {"reply_in": reply_in, "reply_out": reply_out,
                                        "all_calls_in": p_in, "all_calls_out": p_out},
            "cost_10k_reply_only_usd": round(CONVERSATIONS * (reply_in * PRICE_IN + reply_out * PRICE_OUT), 2),
            "cost_10k_all_calls_usd": round(CONVERSATIONS * (p_in * PRICE_IN + p_out * PRICE_OUT), 2),
            "probe_turn_mean_prompt_tokens": round(statistics.mean(
                r["prompt_tokens"] for r in allr if r["phase"] == "probe"), 1),
        }
    return out
```

Setting	Input tokens / conv.	Output tokens / conv.	Cost, reply only	Cost, all calls
SHORT_TERM_N=4 (best)	21,223	2,982	$199.68	$361.33
SHORT_TERM_N=12	24,751	2,586	$241.41	$376.81
SHORT_TERM_N=30 (worst)	34,346	2,497	$328.69	$468.31
Compression policy	27,783	2,182	$258.67	$386.93

Token columns count all calls (reply, fact extraction and summaries). Reply only = the chat reply call alone.

- Best setting: SHORT_TERM_N=4. All settings tied on probe quality (5.00), so the best is the cheapest setting at the top quality: $361.33 for 10,000 conversations.
- Worst setting: SHORT_TERM_N=30. Highest cost ($468.31) with no gain in recall.
- Difference: $106.98 per 10,000 conversations (about 30% more for N=30). Input tokens are most of the cost, so trimming the window matters more than shortening replies.

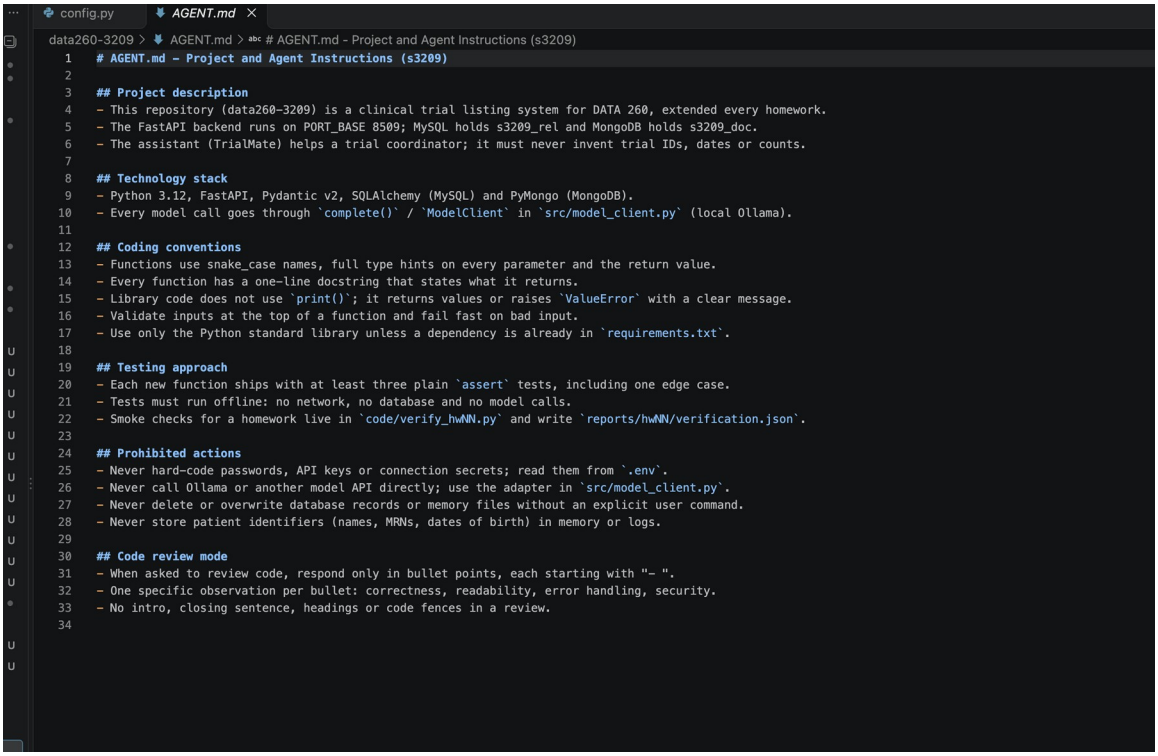
Part 4 - File-Based Memory and Consolidation

Second memory system: AGENT.md (instructions) and MEMORY.md (durable facts) in the repository root, loaded into the system prompt of a command-line chat (code/hw6_memory_cli.py). MEMORY.md line format: - [date] key: value {src=protocol|team|chat}.

1. Memory Files

Code 1 - Final AGENT.md

20 instructions in 6 sections: project description, technology stack, coding conventions, testing approach, prohibited actions, code review mode.



2. File-Based Memory Functions

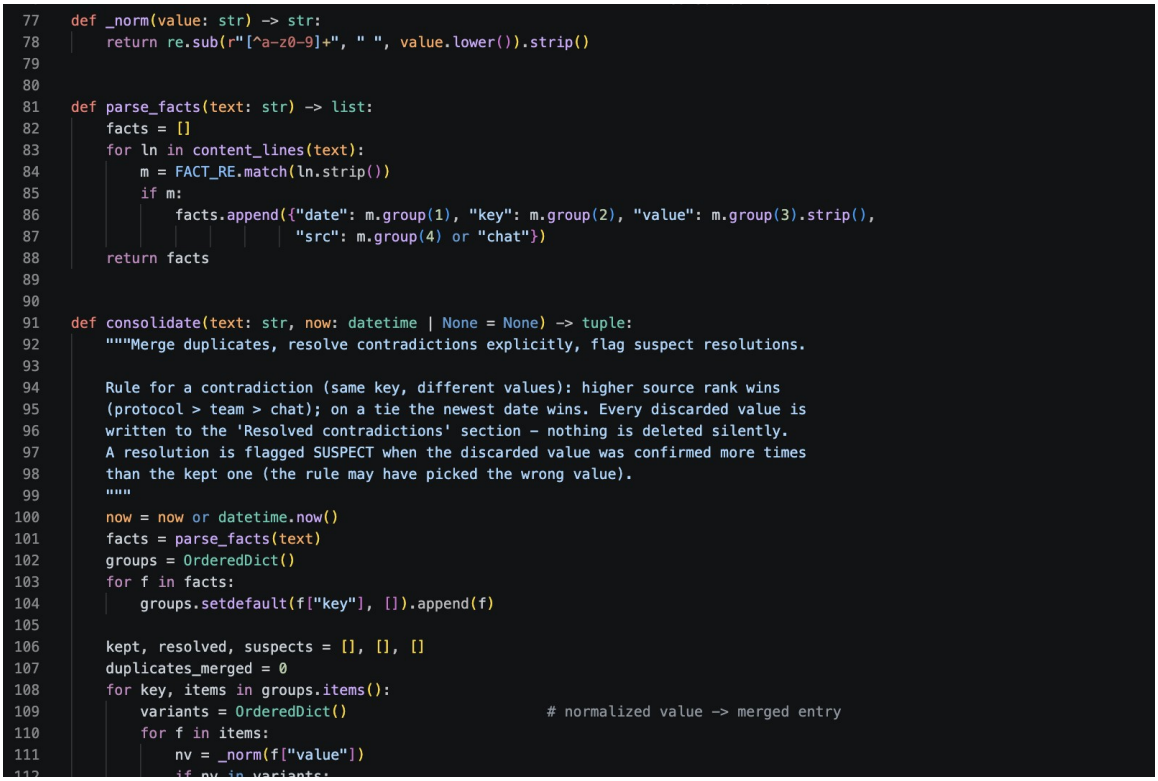
Code 2 - Load files, build prompt, consolidation checks (file_memory.py)

load_memory_files, build_system_prompt, read_last_consolidated, and needs_consolidation (more than 40 non-empty, non-comment lines, or last consolidation older than 24 hours).



Code 3 - Consolidation, part 1 (file_memory.py)

Facts parsed and grouped by key; resolution rule in the docstring.



```python
def _norm(value: str) -> str:
    return re.sub(r"[^a-z0-9]+", " ", value.lower()).strip()


def parse_facts(text: str) -> list:
    facts = []
    for ln in content_lines(text):
        m = FACT_RE.match(ln.strip())
        if m:
            facts.append({"date": m.group(1), "key": m.group(2), "value": m.group(3).strip(),
                          "src": m.group(4) or "chat"})
    return facts


def consolidate(text: str, now: datetime | None = None) -> tuple:
    """Merge duplicates, resolve contradictions explicitly, flag suspect resolutions.

    Rule for a contradiction (same key, different values): higher source rank wins
    (protocol > team > chat); on a tie the newest date wins. Every discarded value is
    written to the 'Resolved contradictions' section — nothing is deleted silently.
    A resolution is flagged SUSPECT when the discarded value was confirmed more times
    than the kept one (the rule may have picked the wrong value).
    """
    now = now or datetime.now()
    facts = parse_facts(text)
    groups = OrderedDict()
    for f in facts:
        groups.setdefault(f["key"], []).append(f)

    kept, resolved, suspects = [], [], []
    duplicates_merged = 0
    for key, items in groups.items():
        variants = OrderedDict()                    # normalized value -> merged entry
        for f in items:
            nv = _norm(f["value"])
            if nv in variants:
```

Code 4 - Consolidation, part 2

Duplicates merged (count kept); contradictions resolved by source rank, then date; every dropped value written to a Resolved contradictions section; SUSPECT flag when the dropped value had more confirmations.

```
    for key, items in groups.items():
        variants = OrderedDict()                        # normalized value -> merged entry
        for f in items:
            nv = _norm(f["value"])
            if nv in variants:
                v = variants[nv]
                v["count"] += 1
                duplicates_merged += 1
                v["date"] = max(v["date"], f["date"])
                if SOURCE_RANK[f["src"]] > SOURCE_RANK[v["src"]]:
                    v["src"] = f["src"]
            else:
                variants[nv] = dict(f, count=1)
        vals = list(variants.values())
        winner = max(vals, key=lambda v: (SOURCE_RANK[v["src"]], v["date"]))
        kept.append(winner)
        for loser in vals:
            if loser is winner:
                continue
            rule = "higher source" if SOURCE_RANK[loser["src"]] != SOURCE_RANK[winner["src"]] else "newer date"
            entry = {"key": key, "kept": winner["value"], "kept_date": winner["date"],
                     "kept_src": winner["src"], "kept_count": winner["count"],
                     "dropped": loser["value"], "dropped_date": loser["date"],
                     "dropped_src": loser["src"], "dropped_count": loser["count"], "rule": rule}
            resolved.append(entry)
            if loser["count"] > winner["count"]:
                suspects.append(entry)

    lines = ["# MEMORY.md - durable facts (s3209)",
             f"<!-- last_consolidated: {now.isoformat(timespec='seconds')} -->",
             "<!-- format: - [YYYY-MM-DD] key: value {src=protocol|team|chat} -->", "",
             "## Facts"]
    for v in kept:
        note = f" <!-- merged x{v['count']} -->" if v["count"] > 1 else ""
        lines.append(f"- [{v['date']}] {v['key']}: {v['value']} {{src={v['src']}}}{note}")
    if resolved:
        lines += ["", "## Resolved contradictions"]
        for r in resolved:
            flag = " SUSPECT: dropped value had more confirmations" if r in suspects else ""
            lines.append(f"- {r['key']}: kept \"{r['kept']}\" ({r['kept_src']}, {r['kept_date']}) over "
                         f"\"{r['dropped']}\" ({r['dropped_src']}, {r['dropped_date']}) - {r['rule']}{flag}")
    new_text = "\n".join(lines) + "\n"
    report = {"lines_before": len(content_lines(text)), "lines_after": len(content_lines(new_text)),
              "facts_before": len(facts), "facts_after": len(kept),
              "duplicates_merged": duplicates_merged, "contradictions_resolved": len(resolved),
              "suspect_resolutions": suspects, "resolutions": resolved}
    return new_text, report
```

3. Memory Commands

Code 5 - /help text and /memory (hw6_memory_cli.py)

Command list and the /memory display (lines, facts, last consolidation, whether consolidation is due).

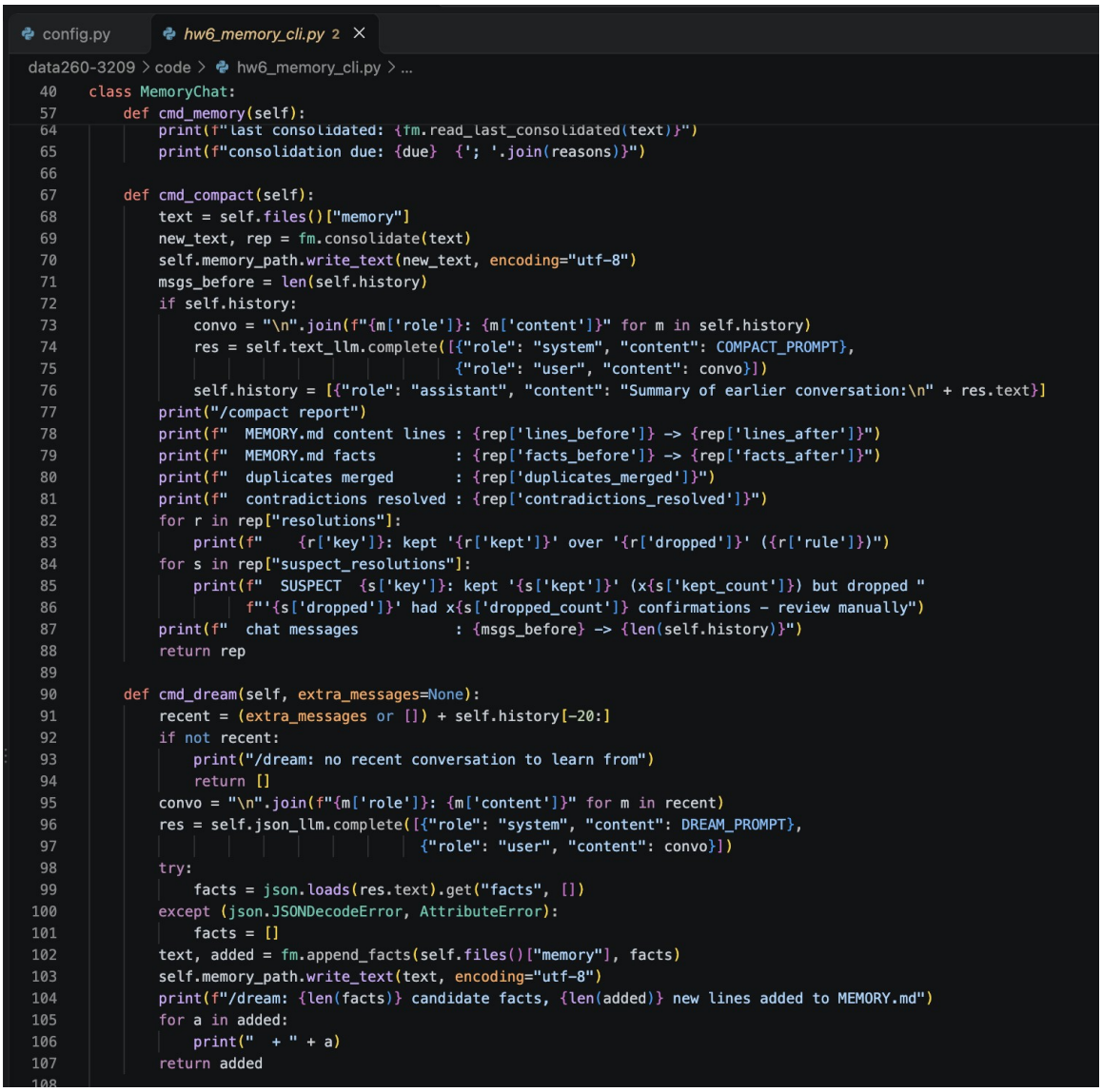
config.py | hw6_memory_cli.py 2

data260-3209 > code > hw6_memory_cli.py > ...

```
23  HELP = """Commands
24    /memory   show MEMORY.md (content lines, last consolidation, whether consolidation is due)
25    /compact  consolidate MEMORY.md (merge duplicates, resolve contradictions) and compact the
26              chat history into one summary message; prints before/after counts
27    /dream    extract durable facts from the recent conversation and append them to MEMORY.md
28    /help     show this help
29    /exit     quit
30  Anything else is sent to the model with AGENT.md + MEMORY.md in the system prompt."""
31
32  DREAM_PROMPT = (
33      "From the conversation below, list durable facts worth remembering across sessions "
34      "(project settings, names, IDs, numbers, dates, decisions). Skip questions and small talk. "
35      'Return JSON only: {"facts": [{"key": "<area.name, snake_case>", "value": "<short value>"}]}'
36  )
37  COMPACT_PROMPT = "Summarize this conversation in at most 5 bullets ('- ' prefix), keeping concrete facts."
38
39
40  class MemoryChat:
41      def __init__(self, memory_path: Path, agent_path: Path):
42          self.memory_path, self.agent_path = memory_path, agent_path
43          self.history = []
44          self.text_llm = ModelClient(model=MODEL, force_json=False, temperature=0.0, seed=3209,
45                                      num_ctx=8192, num_predict=300)
46          self.json_llm = ModelClient(model=MODEL, force_json=True, temperature=0.0, seed=3209,
47                                      num_ctx=8192, num_predict=400)
48
49      # -- helpers
50      def files(self):
51          return fm.load_memory_files(self.agent_path, self.memory_path)
52
53      def system_prompt(self):
54          return fm.build_system_prompt(self.files(), BASE_PROMPT)
55
56      # -- commands
57      def cmd_memory(self):
58          text = self.files()["memory"]
59          due, reasons = fm.needs_consolidation(text)
60          print(text.rstrip())
61          print("-" * 60)
62          print(f"file: {self.memory_path.name}")
63          print(f"content lines: {len(fm.content_lines(text))}  facts: {len(fm.parse_facts(text))}")
64          print(f"last consolidated: {fm.read_last_consolidated(text)}")
65          print(f"consolidation due: {due}  {'; '.join(reasons)}")
66
```

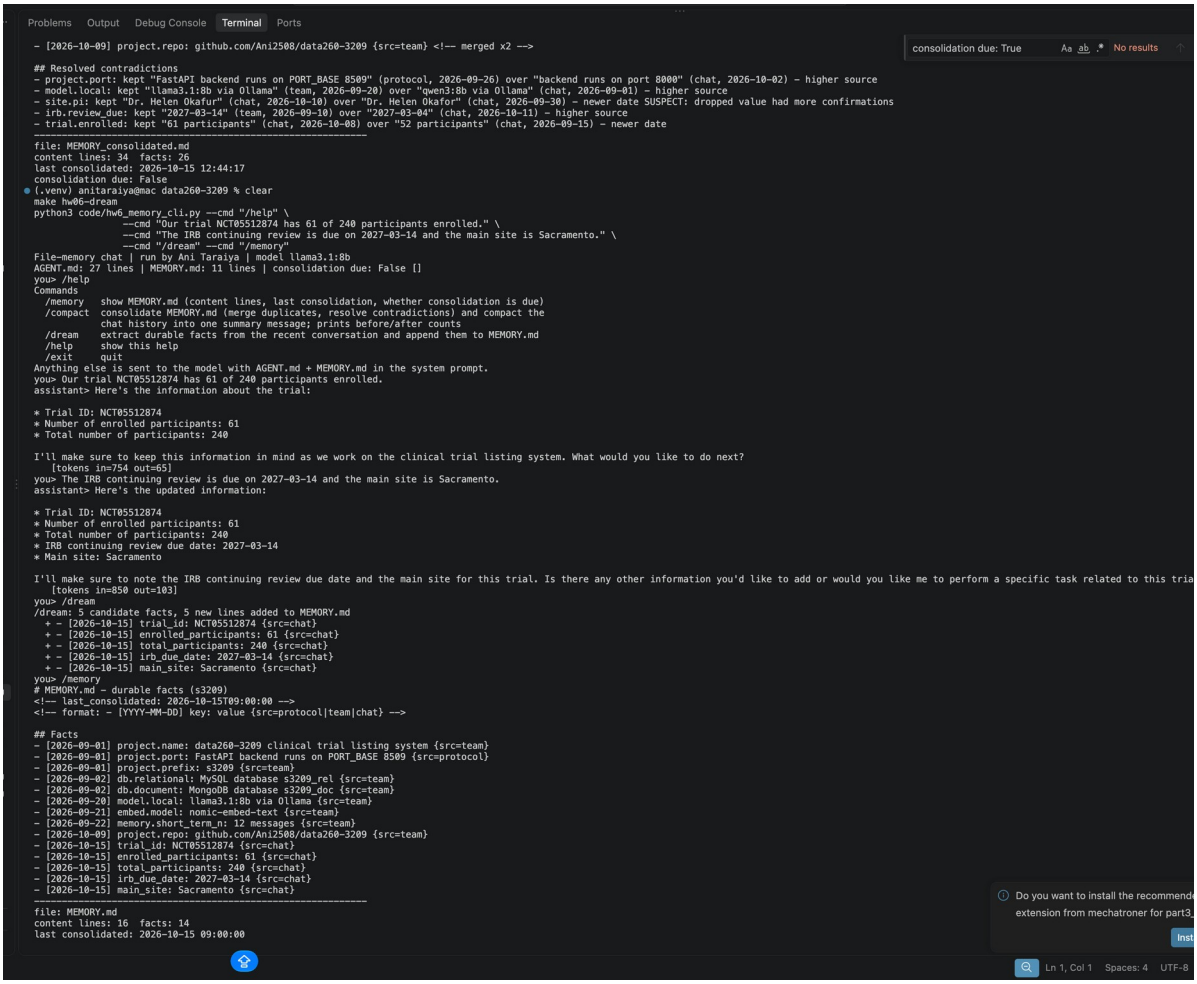
Code 6 - /compact and /dream

/compact consolidates MEMORY.md and compacts chat history, printing before and after counts; /dream extracts facts from recent chat and appends new ones.



Output 1 - /help and /dream (make hw06-dream)

/dream found 5 facts in the chat and added 5 new lines; /memory then shows 14 facts.



4. AGENT.md Verification

Code 7 - Rule checks (hw6_agent_check.py)

The function is generated with AGENT.md as the system prompt, then checked rule by rule with ast and by running its tests offline plus three independent cases.

```python

def run_checks(code: str) -> list:
    tree = ast.parse(code)
    funcs = [n for n in tree.body if isinstance(n, ast.FunctionDef)]
    main = next((f for f in funcs if f.name == "days_until_irb_review"), None)
    checks = []

    def add(rule, ok, detail):
        checks.append({"agent_md_rule": rule, "pass": bool(ok), "detail": detail})

    add("snake_case function names", funcs and all(re.fullmatch(r"[a-z_][a-z0-9_]*", f.name) for f in funcs),
        [f.name for f in funcs])
    if main is None:
        add("function exists", False, "days_until_irb_review not found")
        return checks
    args = main.args.args
    add("type hints on every parameter and the return value",
        all(a.annotation is not None for a in args) and main.returns is not None,
        {a.arg: ast.unparse(a.annotation) if a.annotation else None for a in args}
        | {"return": ast.unparse(main.returns) if main.returns else None})
    doc = ast.get_docstring(main) or ""
    add("one-line docstring that states what it returns",
        len(doc.strip().splitlines()) == 1 and "return" in doc.lower(), doc.strip()[:120])
    prints = [n for n in ast.walk(main) if isinstance(n, ast.Call) and getattr(n.func, "id", "") == "print"]
    add("no print() in library code", not prints, f"{len(prints)} print call(s)")
    raises = [n for n in ast.walk(main) if isinstance(n, ast.Raise)]
    raises_value = any("ValueError" in ast.unparse(r) for r in raises)
    add("validates input and raises ValueError", raises_value, f"{len(raises)} raise statement(s)")
    imports = set()
    for n in ast.walk(tree):
        if isinstance(n, ast.Import):
            imports |= {a.name.split(".")[0] for a in n.names}
        elif isinstance(n, ast.ImportFrom) and n.module:
            imports.add(n.module.split(".")[0])
    add("standard library only / no direct model API", imports <= STDLIB and not imports & {"ollama", "requests"},
        sorted(imports))
    asserts = [n for n in ast.walk(tree) if isinstance(n, ast.Assert)]
    ut = [n for n in ast.walk(tree) if isinstance(n, ast.Attribute) and n.attr.startswith("assert")]
    add("at least three plain assert tests", len(asserts) >= 3,
        f"{len(asserts)} plain assert(s), {len(ut)} unittest assert call(s)")
    secrets = re.findall(r"(?i)(password|api_key|secret)\s*=", code)
    add("no hard-coded secrets", not secrets, secrets)
    return checks


def run_tests(path: Path) -> dict:
```

Output 2 - Attempt 1: generated function (make hw06-agent-check)

days_until_irb_review generated by llama3.1:8b with AGENT.md in the system prompt.

```
Problems  Output  Debug Console  Terminal  Ports

(.venv) anitaraiya@mac data260-3209 % clear
make hw06-dream
file: MEMORY.md
content lines: 16  facts: 14
last consolidated: 2026-10-15 09:00:00
consolidation due: False
(.venv) anitaraiya@mac data260-3209 % clear
make hw06-agent-check
python3 code/hw6_agent_check.py
AGENT.md compliance check | run by Ani Taraiya | model llama3.1:8b
======================================================================
Attempt 1: generated_function_attempt1.py
----------------------------------------------------------------------
from datetime import datetime, timedelta

def days_until_irb_review(due_date: str, today: str) -> int:
    """
    Calculate the number of days until the IRB continuing review is due.

    Args:
    - due_date (str): ISO date string (YYYY-MM-DD) of the IRB continuing review due date.
    - today (str): ISO date string (YYYY-MM-DD) of the current date.

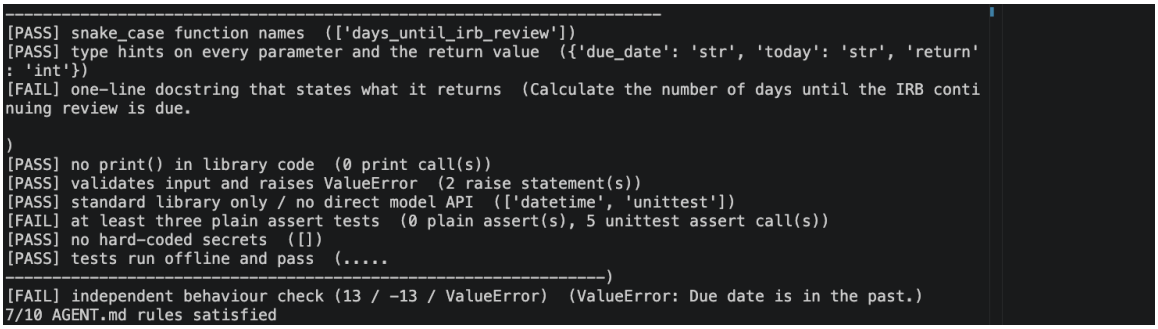
    Returns:
    - int: The number of days from today until the IRB continuing review is due (negative if overdue).
    """
    try:
        due_date = datetime.fromisoformat(due_date)
        today = datetime.fromisoformat(today)
    except ValueError as e:
        raise ValueError("Invalid date format. Please use YYYY-MM-DD.") from e

    if due_date < today:
        raise ValueError("Due date is in the past.")

    return (due_date - today).days
```

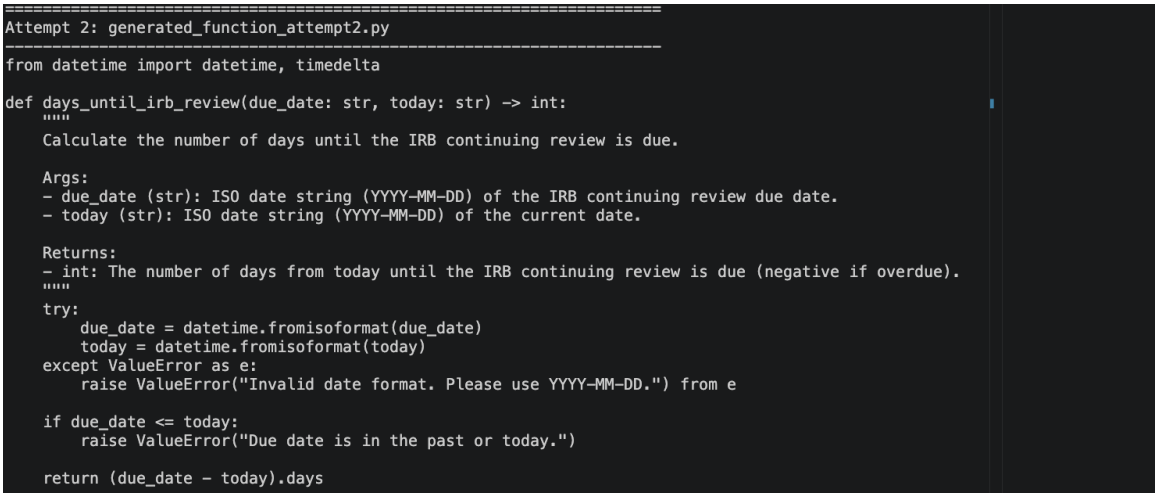
Output 3 - Attempt 1: rule results

7/10 rules satisfied.



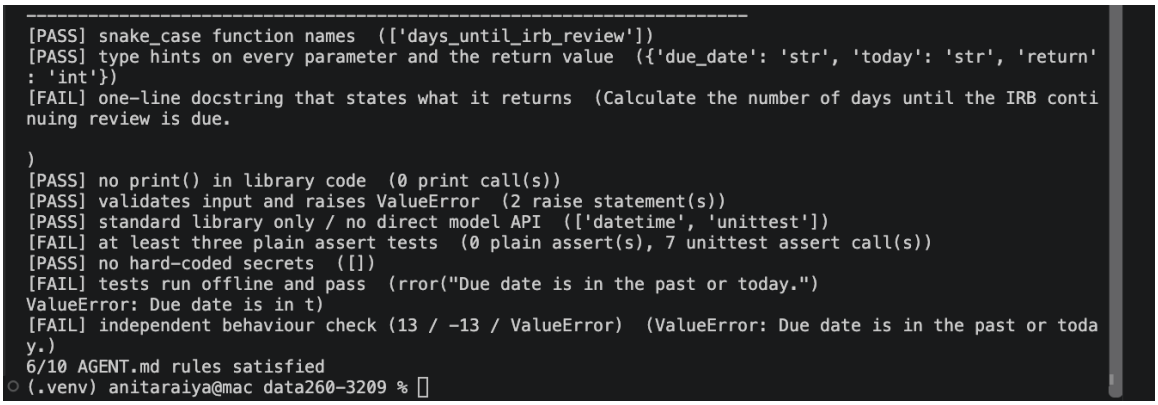
Output 4 - Attempt 2 after feedback: generated function

The failed rules were sent back to the model; it changed < to <= only.



Output 5 - Attempt 2: rule results

6/10 rules satisfied; its own tests now fail.



AGENT.md rule	Attempt 1	Attempt 2
snake_case, type hints, no print()	Pass	Pass
Raises ValueError, standard library only, no secrets	Pass	Pass
One-line docstring	Fail (10-line docstring)	Fail

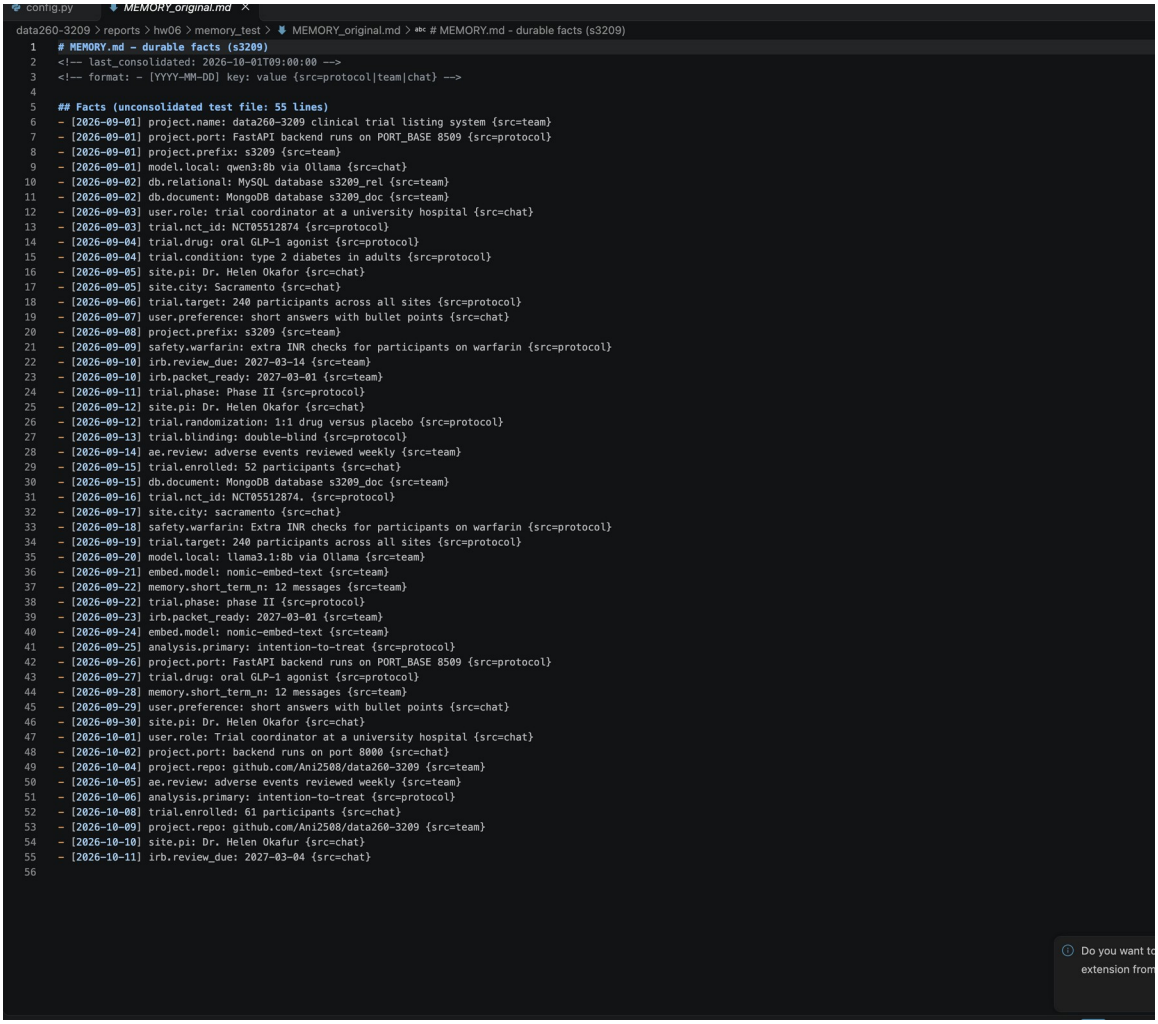
At least three plain assert tests	Fail (unittest instead)	Fail
Tests run offline and pass	Pass	Fail
Task behaviour: overdue gives a negative number	Fail (raises error)	Fail
Total	7/10	6/10

- Result: the generated function does not fully follow AGENT.md. It keeps the naming, typing, error and dependency rules, but ignores the docstring and test rules and the negative-days requirement.
- One feedback round did not fix it and broke its own tests. The checker caught every violation, so AGENT.md is useful as a testable spec, but its rules need automatic checks rather than trust.

5. Consolidation Test

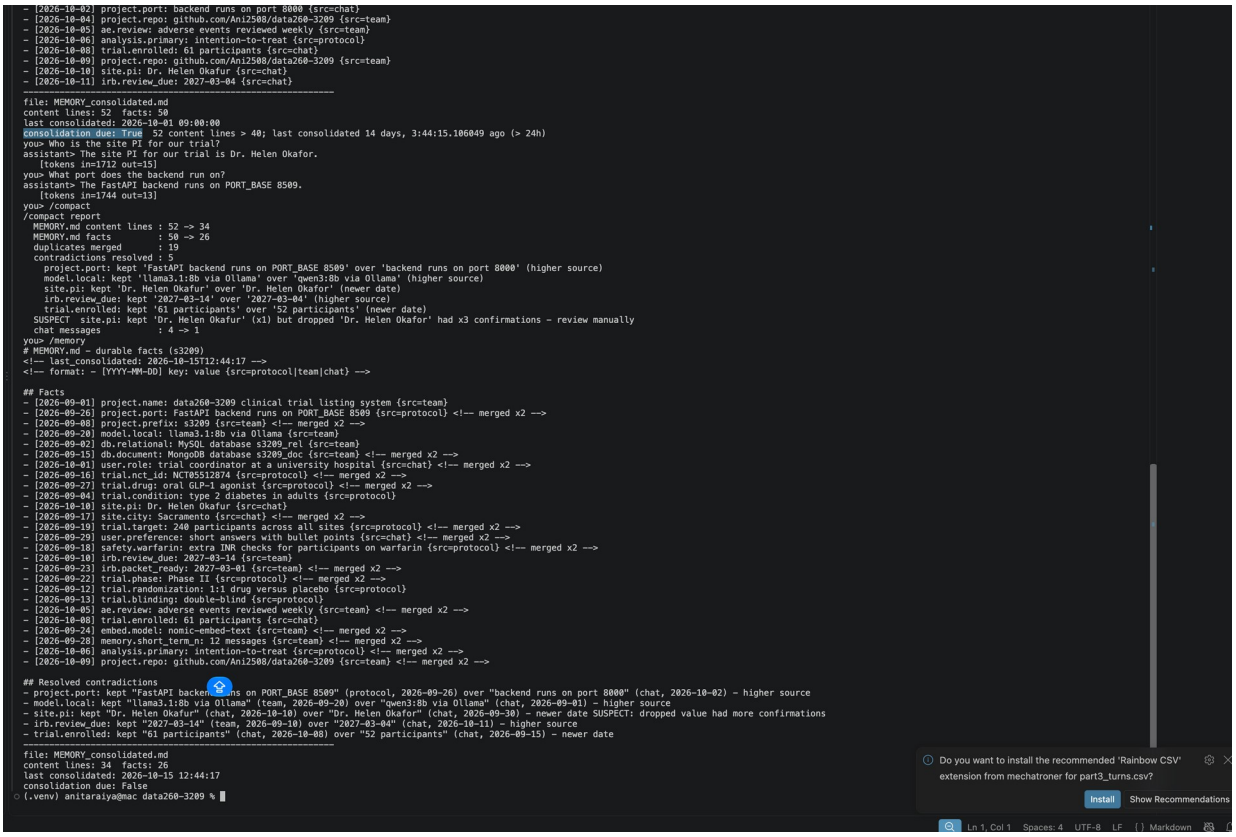
Output 6 - MEMORY_original.md (55 lines)

Duplicates (for example project.prefix, safety.warfarin, site.pi x3) and contradictions (project.port, model.local, site.pi, irb.review_due, trial.enrolled).



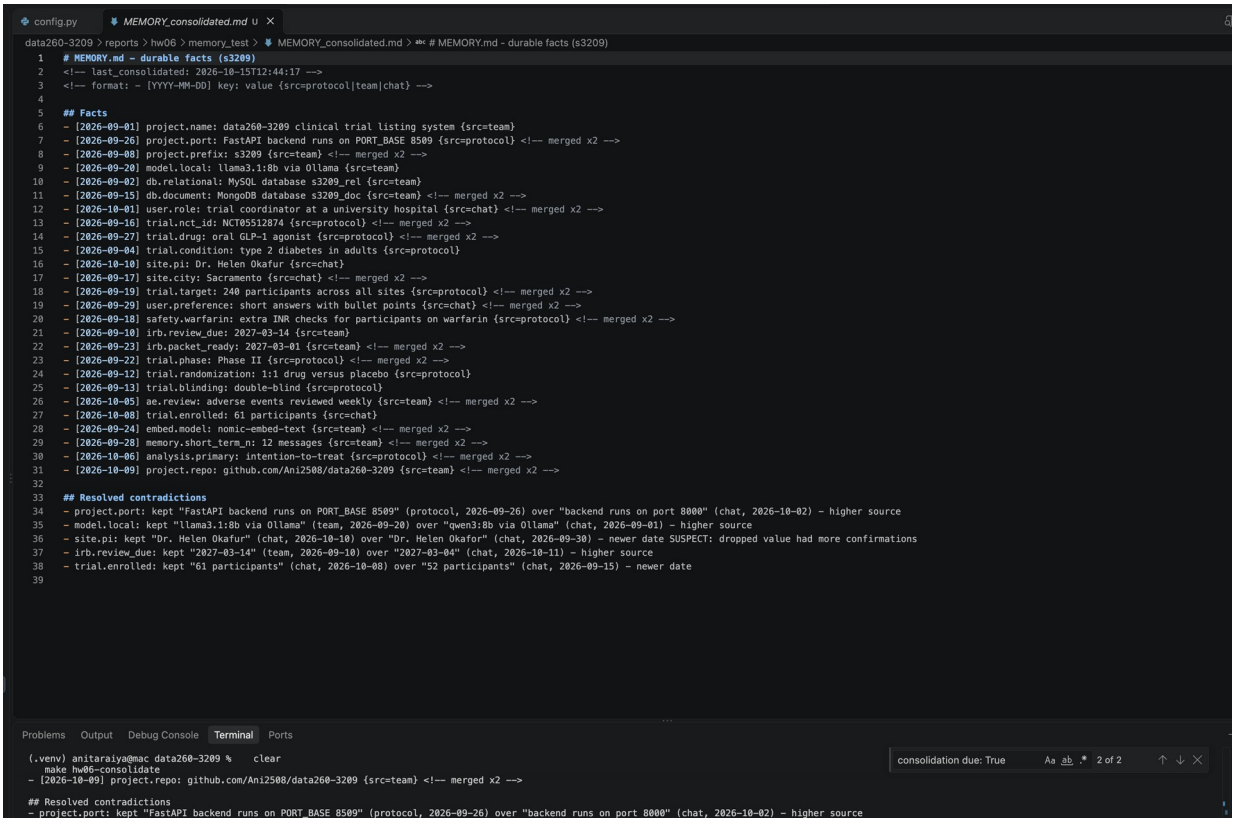
Output 7 - /compact before and after (make hw06-consolidate)

consolidation due: True (52 content lines, 14 days old); after /compact: 52 -> 34 content lines, 50 -> 26 facts, 19 duplicates merged, 5 contradictions resolved, chat messages 4 -> 1.



Output 8 - MEMORY_consolidated.md (38 lines)

Merged facts plus an explicit Resolved contradictions section; nothing deleted silently.



Key	Kept	Dropped	Rule	Correct?
project.port	PORT_BASE 8509 (protocol)	port 8000 (chat)	higher source	Yes
model.local	llama3.1:8b (team)	qwen3:8b (chat)	higher source	Yes
irb.review_due	2027-03-14 (team)	2027-03-04 (chat)	higher source	Yes
trial.enrolled	61 (2026-10-08)	52 (2026-09-15)	newer date	Yes
site.pi	Okafur (x1, newer)	Okafor (x3)	newer date	No - flagged SUSPECT

Incorrect resolution: site.pi kept the newer spelling "Okafur", which is a typo; "Okafor" was confirmed three times. The newest-date rule picked the wrong value, but the SUSPECT flag marks it for manual review and the correct value is kept in the file. Both versions are committed: reports/hw06/memory_test/MEMORY_original.md and MEMORY_consolidated.md.

6. MongoDB Memory vs File-Based Memory

Criterion	MongoDB memory (Parts 2-3)	File-based memory (AGENT.md + MEMORY.md)
Inspectability	Needs Compass or queries; embeddings are not readable	Plain Markdown; readable in any editor and diffable in Git

Persistence after restart	Persists in the database server; needs MongoDB running	Persists on disk and in Git history; no server needed
Scalability across many sessions	Indexed per user and session; handles many users and sessions	One shared file; grows with every fact, needs consolidation, does not scale to many users
Facts that must not be forgotten	Risky: summaries can drop details; retrieval is top-k only	Better: always loaded in full and edited on purpose; reviewed through Git

- Best use: MongoDB for per-user conversation memory at scale; MEMORY.md and AGENT.md for a small set of project rules and critical facts that must always be in the prompt.

AI Use

1. What I used an AI assistant for and what I did myself. I used an AI assistant to learn and understand the concepts in this homework: the MongoDB document API, the memory chat logic (short-term window, summaries, episodic facts), the Part 3 experiment and scoring, the file-based memory commands, and the verification script. It also helped write the code for these parts and helped arrange the report layout and check grammar. I did the setup and running myself. I installed MongoDB and Compass on my Mac, ran the backend, Ollama and every experiment and command on my machine, took all the screenshots, checked the outputs against the requirements and decided what to change when results looked wrong.

2. One AI-produced output that was wrong. The first version of the episodic memory extraction stored facts the user never said. In my Part 2 run, the episodes collection held 20 facts, including "You requested a JSON output", "You asked me to summarize your trial", and made-up advice such as "Submit report within 24 hours of occurrence". These pushed the real facts out of the top-3 retrieval.

3. How I found it. I read the JSON from GET /api/memory/ani_demo and the episodes collection in Compass after the 12-message run. Many facts did not match anything I typed, and the facts retrieved for the warfarin and IRB messages were screening-visit advice instead of the details I had just given.

4. What I changed and why it works now. The extraction prompt now asks only for facts the user states, copied exactly, with one positive and one negative example. Messages ending with "?" are skipped and facts like "You asked..." are filtered out. Embeddings use separate query and document prefixes. After rerunning Part 2, the session stored 9 facts, all from my messages, and the site/PI question retrieved the correct two facts.